

Getting Started

Installing SD Core for Windows, and finding your way around it for the first time.

SD CORE FOR WINDOWS W1.0-0

Copyright and licence

Who wrote this set, how it is licensed, and what the licence lets you do with it.

This page carries the copyright and the licence for **the whole GettingStarted set**. Every other page in the set names them in one line at the foot; the full terms are here, once, so that they are stated in one place and cannot come to disagree with each other.

RELEASED WITH	SD Core for Windows W1.0-0
COPYRIGHT	Copyright © 2026 Donald Montaine
LICENCE	Creative Commons Attribution-ShareAlike 4.0 International (CC BY-SA 4.0)

You are free to **share** this document - copy and redistribute it in any medium or format - and to **adapt** it - remix, transform and build upon it - for any purpose, including commercially.

Two conditions apply. **Attribution:** you must give appropriate credit, provide a link to the licence, and indicate if changes were made. **ShareAlike:** if you remix, transform or build upon this document, you must distribute what you produce under the same licence. You may not add legal terms or technological measures that restrict others from doing anything the licence permits.

This is a summary and not a substitute for the licence itself. The complete text is at <https://creativecommons.org/licenses/by-sa/4.0/>.

The other two sets

The documentation is three separate sets, and each is handed out on its own: **GettingStarted**, **User** and **Administrator**. Each carries its own copy of this page, under the same copyright and the same licence.

The set this page belongs to

Read Start here for what the GettingStarted set covers and the order to read it in.

Start here

What this set covers, who it is for, and what it deliberately leaves out.

You already know MultiValue. This set does not teach it.

SD Core for Windows is a version of SD, with elements found in the main SD version and in ScarletDME. ScarletDME was a fork of the original GPL release of OpenQM 2.6.6.

That lineage matters when you go looking for documentation. Not all the features of the **commercial** OpenQM 2.6.6 were in the GPL release, and **no documentation specific to the GPL version was ever released**. So even the OpenQM documents are not authoritative here.

The OpenQM 2.6.6 documents can be used as a reference, but SD Core has additions, changes and deletions — of features, of structure, of security and of commands. **These pages cover those changes.**

If you have used OpenQM, or SD on Linux, much of SD Core will still be familiar: the same data model, the same query processor, the same BASIC.

What "used to" means in these pages

These pages describe changes, so **used to**, **no longer** and **now** run all through them. **The comparison is against the code this version was made from** — SD on Linux, and ScarletDME and OpenQM 2.6.6 behind it.

It never means an earlier release of SD Core for Windows, because this is the first one. Some of the changes are Windows-only and have no Linux original to differ from; those say **earlier builds of this port** instead, so the two are never confused.

The pages

There are nineteen, and they are numbered. Read them in numerical order the first time; after that they stand alone.

A number with a letter after it is the second half of a long page, split so that no page runs longer than a reader will scroll: `01a` continues `01`, and so on. Nothing was renumbered when they were split.

00	Start here	this page — what the set is and what it leaves out
00a	Copyright and licence	The copyright and the licence for this set, in full and in one place
01	Installing SD Core	What the installer does, the two kinds of installation, and the choices it offers

01a	Upgrading and uninstalling	Installing a new release over an existing one, and taking SD off the machine
02	Your first thirty minutes	Start here if you just want it working — install to a second user signing in, in eight steps
03	Running SD	The service, starting and stopping, and recovering from an unclean shutdown
04	Scheduled jobs	Running an SD command on a timer, and the permit list that decides which ones
05	Account types	Standard, Programmer, Administrator, Suspended and Group — what each one is, and how to make one
05a	Managing accounts	Group accounts, sharing one, changing an account afterwards, and deleting it
06	Administrator commands	The verbs an administrator account gets, and how to use them
07	Programmer commands	The development verbs a standard account does not get
08	ssh access	How people reach SD on this machine, and why it is ssh
09	API access	The client API, its port, and the login that replaced the old one
10	Client distribution	Which library an application needs, and the one file no installer can update
11	Lower case	Case in commands, file names, record ids and account names
12	Security	The identity model, and what protects the database
12a	Security and the operating system	Reaching the machine from inside SD, how privileged work is done, and the audit trail
13	Other hardening	Auditing, the shell permit list, and the rest
14	Not in SD Core	What has been removed, and what to use instead

The five things most likely to surprise you

1. Signing in asks for no password. Windows has already authenticated you. `sd` puts you in the SD account with your own name; if there is no such account, or you are not in the `sdusers` group, you are refused. Administration is gated on being an elevated Windows administrator, not on a secret SD holds. See Security.

2. Accounts SD creates cannot log in to Windows at this machine. They are denied the physical console and Remote Desktop, deliberately. They reach SD **over ssh, or through an API client, or both** — and which of those is a required keyword on `create.account`. **Multi-user access over Remote Desktop is not supported** and is not a gap to be filled later. See ssh access and API access.

3. A new account gets a reduced VOC unless you say otherwise. A standard account cannot compile, catalogue, edit or create files. `programmer` and `administrator` keywords on `create.account` decide that. See Account types.

4. Commands and names are lower case now. Everything that can be lower case is. Typing in upper case still works — the lookup tries what you typed, then lower, then upper. See Lower case.

5. The API login is SCRAM, and the old cleartext one is gone. Clients built against the old protocol will not connect. See API access.

What this release is

W1.0-0. Windows only. There are no `#ifdef` branches keeping Linux alive in this source — Linux SD is a separate project and this is not a build of it.

It is a hobby project with no release schedule.

This set is the delta. It covers installing SD Core on Windows, running it, and what differs from OpenQM and from SD on Linux. The reference for the language and the command processor is a separate set, and so is the administrator's.

Where the source is

Both repositories are public, and everything in them is open source. SD is GPL software — `config gpl` at an `sd` prompt displays the licence, and the installed tree carries it as a file.

The server, the client libraries and the installer	https://github.com/dmontaine/sd4windows
--	---

These pages	https://github.com/dmontaine/SDCoreWindowsDocs
-------------	---

Neither repository contains a built binary, deliberately — no `.exe`, no `.dll`, no object files. A clone builds. That is why installing means building, and why there is no download of a compiled artefact in the repository itself.

The documentation repository holds the **Markdown only**; the HTML and PDF you are reading are generated from it and are not stored there.

Reporting what you find

Open an issue on the server repository — <https://github.com/dmontaine/sd4windows/issues> — for anything about SD itself, and on the documentation repository for an error in these pages. If you are not sure which, the server one is the right guess.

Issues rather than pull requests. Both repositories are readable by anyone and **writable only by the author**, so a change cannot be merged from outside; a clear issue is worth more than a patch nobody can apply. A patch attached to an issue is welcome, it just travels that way.

The two things worth reporting in most detail are **anything that behaves differently from OpenQM and is not described here**, and **anything in these pages that turns out not to be true of the build you are running**. The second is as valuable as the first.

Quote the version as `w1.0-0` — the string in the header bar of every page here, in the installer's file name, and in what `sd --version` reports. The bare `1.0-0` is the same release; the `w` says it is the Windows one, and that is the part worth keeping in a report.

Installing SD Core

What the installer does to the machine, the two kinds of installation, and the choices it puts in front of you.

There is no compiler to run and no dependency to resolve. SD Core for Windows ships as a single `sd-setup-w1.0-0.exe`, carries its own runtime beside `sd.exe`, and installs in one pass.

This is the largest single difference from the regular SD version — that, and the fact that this version is for Windows and not Linux. In regular SD, `installsdai.sh` existed because ScarletDME targeted four distributions and the end user had to compile; Windows has one target and one ABI, so none of that transfers.

Before you start

You need an elevated session. The installer creates local groups, sets file permissions, registers a service and assigns user rights. You can start the install from a normal console session — you will get an elevation prompt.

SD cannot be installed silently. This is deliberate, not a missing feature. The installer asks questions whose answers cannot be defaulted safely — which kind of installation, whether to expose ports, and the password on the account it makes for you.

The installer does not ask where to put SD. Both roots are fixed. See What lands where.

IT CHECKS THE MACHINE BEFORE IT CHANGES ANYTHING

The installer refuses to start, and changes nothing, if it finds:

- another ssh server using port 22, or
- an ssh service that is not part of Windows, installed even if it is not running, or
- Windows' own ssh server with settings somebody has already changed.

It tells you what it finds — the service and its path, or the directive that was added — so you can decide what to do. You may **remove the other ssh server, or put the ssh configuration back the way Windows shipped it, and run the installer again.**

Why it is this strict: accounts SD creates cannot log in to Windows at all, so ssh is one of only two ways they reach the machine — and SD configures the ssh server so those sessions land inside SD and cannot reach a command prompt. **SD can only promise that about a server it installed and configured itself.**

What you are asked

There is one kind of installation. What varies is how the machine can be reached afterwards, and that is three tick boxes on one page.

The boxes appear only on a **first** install. An upgrade shows no tasks page at all: the machine already carries the answers, and every one of these settings has a command that changes it afterwards.

1. SYSTEM INTEGRATION

	DEFAULT
Add SD Core to the system PATH, so <code>sd</code> runs from any directory	ticked

Changed afterwards with `append.sd.path on` or `off`.

2. THE SSH SERVER

Installing an ssh server is optional, and the box is not ticked. It downloads from Windows Update and can take several minutes — up to about an hour on a slow machine or connection — which is why it is offered rather than done.

What you see depends on what the machine already has:

MACHINE	BOXES	DEFAULTS
No ssh server	<i>Install the OpenSSH server, and indented under it <i>Let other computers connect</i></i>	both unticked
Server present, firewall shut	<i>Let other computers connect</i>	unticked
Server present, firewall open	<i>Let other computers connect</i>	ticked

The second box is a child of the first, so Windows greys it out until the parent is ticked. You cannot ask for remote ssh access without an ssh server.

The last row matters on a machine that already uses ssh: the box shows the **current** firewall scope, so leaving it alone changes nothing in either direction. `OpenSSH-Server-In-TCP` is Windows' own shared rule and not SD's, so defaulting it to unticked and applying that would silently loopback-lock the ssh a site already relies on.

Changed afterwards with `ssh.server install` or `remove`, and `remote.ssh on` or `off`.

3. THE API

	DEFAULT
Provide the SD Core API (port 4243)	unticked
Let other computers on your network reach it	unticked

The API is off unless you ask for it. Decline it and SD is installed with a configuration carrying no `APIPORT` line, so no socket is opened at all — not a listener behind a closed firewall, but no listener.

Again the second box is a child of the first. The pair appears only when there is no existing configuration file to read the answer from.

Changed afterwards with `remote.api on`, `local` or `off`.

AN INSTALLATION WITH NEITHER IS A SUPPORTED CHOICE

Leaving every remote box unticked gives a working SD that nothing outside the machine can reach. That is a real deployment rather than a degraded one.

Be aware of what it means for accounts. Accounts SD creates are denied the Windows console and Remote Desktop, so they reach SD over ssh or over the API and nothing else — including at this keyboard, where an ssh account arrives by `ssh localhost`. With no ssh server on the machine, `create.account ... ssh` and `... both` are refused, because the account could sign in nowhere. `api` and `none` still work, and `none` is meaningful: an application account reached only by `logto`.

The refusal is tested against the machine when you type the command, not against a decision recorded at install time. Install an ssh server later and `create.account ... ssh` simply starts working.

What lands where

WHAT	WHERE	WAS, ON LINUX
Binaries, and the MSYS2 DLLs beside them	<code>C:\Program Files\SD\usr\bin\</code>	<code>/usr/local/bin</code>
The changelog	<code>C:\Program Files\SD\changelog</code>	—
Configuration	<code>C:\ProgramData\SD\sd.conf</code>	<code>/etc/sd.conf</code>
The SDSYS account	<code>C:\ProgramData\SD\sdsys\</code>	<code>/usr/local/sdsys</code>
User accounts	<code>C:\ProgramData\SD\user_accounts\</code>	<code>/home/sd/user_accounts</code>
Group accounts	<code>C:\ProgramData\SD\group_accounts\</code>	<code>/home/sd/group_accounts</code>
POSIX shared memory	<code>C:\ProgramData\SD\shm\</code>	<code>/dev/shm</code>

`usr\bin` is **load-bearing, not tidiness**. Shipping `msys-2.0.dll` beside the executable relocates the POSIX root to the DLL's directory minus two components, so only that depth puts `/` on `C:\Program Files\SD\`. Do not move the binaries.

The DLLs ship beside `sd.exe` on purpose. Do not change their location.

CONFIGURATION

Server and client both read `SD_CONFIG`, then fall back to `%ProgramData%\SD\sd.conf`.

`sd.conf` is installed `onlyifdoesntexist` and marked never to uninstall, so your edits survive both upgrade and removal.

What the installer creates on the machine

<code>sdusers</code>	grants access to the files under <code>C:\ProgramData\SD</code> . Everyone who uses SD needs it
<code>sdsshonly</code>	carries the two deny rights that confine an account to ssh. Every non-administrator account SD creates joins it
<code>sdu_<name></code>	one per account, created by <code>create.account</code>
the service	String Database (SD)
ACLs	inheritance is broken on <code>C:\ProgramData\SD</code> and access granted narrowly. This is what makes the database private from the rest of the machine

You must sign out and back in after being added to `sdusers`. Windows group membership is carried in your logon token. Until you get a new one, you cannot read the data tree at all, and the symptom looks like a broken install.

OpenSSH

The OpenSSH server is installed only if you ask for it, and the box is not ticked. This section covers what happens when you do ask.

Accounts SD creates cannot log in to Windows, so they reach the machine over ssh or through an API client. An installation offering neither is usable by nobody but you — which is a legitimate choice, and worth making knowingly. A local-only machine is served by `ssh localhost`, which needs no network.

It is slow and it looks like a hang. `Add-WindowsCapability` hands off to `TiWorker` and can work for minutes with nothing on screen. **Never kill it** — interrupting Windows servicing mid-flight is how the component store gets corrupted. It may also leave a reboot pending, which is real; SD itself needs none.

If it cannot be installed, the install still succeeds. It is a Features on Demand capability and can be blocked by policy, a WSUS with no source, a metered connection or an offline machine. The installer reports it in as many words, with the command to retry — and you should read that report, because in that state **no account but yours can sign in anywhere.**

It is the same command either way, and it is repeated here because the closing report is easy to close. From an elevated prompt:

```
powershell -File "C:\Program Files\SD\install-ssh.ps1"
```

It exits **0** installed and running, **2** installed but Windows wants a restart before the service exists, **1** failed — and it prints which, so run it in a window you can read rather than expecting a log. It is safe to run twice: on a machine that already has the server it says so and changes nothing.

The full-screen editors

The installer makes sure the two terminal editors are on the machine, because the `edit` and `micro` verbs run them. Current Windows builds already carry `edit.exe`; `micro` never ships with Windows, so it is always a winget install. Neither is offered as a choice, for the same reason the ssh server is not: a programmer account with a verb that does nothing is worse than either answer.

If one cannot be installed, the install still succeeds. No winget, no network or a policy in the way all end the same: SD is complete and one editor verb is not — `ed`, the line editor, needs nothing. What happened is in `C:\ProgramData\SD\install-editors.log`, and to do it by hand afterwards, from an elevated prompt:

```
winget install -e --id Microsoft.Edit --scope machine
winget install -e --id zyedidia.micro --scope machine
```

The scope is not optional. Without it winget installs into the profile of whoever ran it, and accounts SD creates cannot log in to Windows at all — so a per-user copy is one they can never reach.

Changing any of it afterwards

Every choice on the tasks page has a verb that changes it later, and that is why an upgrade does not ask again. All four need an elevated administrator session, and all four report when given no keyword:

<code>ssh.server</code> <code>install</code> <code>remove</code>	add or remove the OpenSSH server
<code>remote.ssh</code> <code>on</code> <code>off</code>	who may reach it
<code>remote.api</code> <code>on</code> <code>local</code> <code>off</code>	whether SD opens its API socket, and who may reach it
<code>append.sd.path</code> <code>on</code> <code>off</code>	whether <code>sd</code> runs from any directory

They are covered in the SD Core for Windows administrator documentation, under *Remote access and the machine*.

`remote.api` `on` and `off` change whether SD opens a socket at all, which it decides at start-up, so they offer to restart SD — and that restart ends every session including the one that asked. `local` and `on` differ only in the firewall and take effect at once.

The underlying scripts are still on the machine and can be run directly if you prefer, from an elevated prompt:

```
powershell -File "C:\Program Files\SD\api-firewall.ps1" -Open
```

At the end

The installer **finishes, and then opens SD** so you can set your own password. It does not leave a wizard page waiting behind the session.

A Start Menu entry, **Check the SD installation**, runs a post-install check you can re-run at any time. It closes on a keypress.

You are told plainly what setting no password costs, rather than being allowed to skip past it silently.

Continued in

Upgrading and uninstalling — upgrading an existing installation, and uninstalling.

Upgrading and uninstalling

Replacing an existing installation, and taking SD off the machine.

This page continues Installing SD Core.

Upgrading

Installing a new release over an existing one updates your database.

REPLACED	KEPT, AND NOT TOUCHED
the catalogue and compiled programs	your accounts and their passwords
the BASIC source	the private catalogue
the messages and include records	which Windows users are linked to which SD accounts
the VOC templates and library routines	the commands each account may run
the SDSYS BP programs	your print queue and held reports
terminfo, the licence, the contributor list	everything under your own accounts, and sd.conf

Anything SD created while it was running — your VOC included — is left exactly as it is.

The dictionaries are brought up to date for you. Upgrading reapplies the dictionary definitions the release ships: it adds and updates the entries SD ships and leaves alone any you added. If that step cannot run, the installer says so at the end rather than finishing quietly, and upgrade-dicts.log in C:\ProgramData\SD says what happened.

Every account's VOC is brought up to date for you. The installer runs update.accounts all, which walks every registered account, so a command this release adds can be typed in accounts that already existed. This did not happen before W1.0-0: an upgrade replaced the shipped files and no existing account — including the system account — ever gained a new verb.

To refresh one account by hand afterwards, update.accounts in that account updates it and offers the rest.

Two limits are worth knowing before you rely on it.

SD only ever adds records to a VOC, never removes them. An account created before a verb was withdrawn keeps it. `update.accounts` cannot be relied on to take something away.

A record you have customised can be held back on purpose. Put `[locked]` in field 1 after the type code and the upgrade leaves that record alone, naming it in a message so you know what was withheld — and therefore which corrections this release made that you have not taken. Verbs are the exception: a locked verb is updated anyway, and you are told which. The administrator documentation covers it under *Accounts and security*.

Uninstalling

It is the standard Windows uninstall — Settings ▸ Apps, or `unins000.exe`.

The default does not touch your accounts, the database or the configuration. Inno removes only what it installed and only removes a directory if it is empty, so everything the running system created is invisible to it.

Removing the data is a separate, opt-in prompt that defaults to keeping it, and says exactly what it destroys and where. **A silent uninstall never deletes the database**, whatever the prompt would have offered.

The uninstaller does not remove OpenSSH. It may predate SD or be in use by something else. It does restore `sshd_config`, keeping the original as `sshd_config.before-sd` — but it deliberately does **not** widen the firewall rule back, because restoring it would mean opening a port on the way out.

Your first thirty minutes

From a finished install to a working account, a file and a second user signing in over ssh.

This page assumes SD Core is installed and you have never used this port before. It is a walkthrough, not a reference — every step links to the page that explains it properly.

Do this first, before anything else.

1. Sign out of Windows and back in

Not optional, and not a tidiness step. The installer put your Windows account into the `sdusers` group, and **Windows fixes group membership when you sign in**. Until you get a new logon token you cannot read the database at all.

The symptom looks like a broken install, not a permissions problem, which is why it is step 1. The post-install check knows this: the run it offers at the end of the installer is **always the incomplete one**, for exactly this reason.

Sign out, sign back in, then run **Check the SD installation** from the Start Menu. It closes on a keypress.

2. Start SD

```
sd
```

You land in **the SD account named after your Windows login**. Nothing asks for a password — Windows has already authenticated you.

SD is already running. It is a Windows service, `String Database (SD)`, and Windows starts it at every boot. You do not type `sd -start`. See Running SD.

If `sd` answers *Account ... not in register*, you are in the wrong account or step 1 has not taken effect. If it answers *not registered for SD use*, you are not in `sdusers`.

3. Look around

```
who  
listf
```

term

who the account you are standing in

listf the files in it

term your terminal type and page size — should say `Device : windows`

If **term** says something else and your arrow keys do not work, run `term windows` for this session and see Other hardening. An account created before the `WINDOWS` definition shipped keeps its old setting until `update.accounts`.

term also reports the page size, and SD's default is 120 × 36 — not 80 × 24. The shipped dictionaries and the default `list` layouts are formatted for 120 columns, so **a console window narrower than that makes ordinary reports look wrapped or truncated** and the report is not at fault. Widen the window, or set it for the session with `term default` — which puts the 120 × 36 back and prints nothing while doing it, so check with a bare `term` afterwards. `term 120,36` is the same thing typed out. See Other hardening.

4. Make a file and put something in it

```
create.file customers
ed customers 1001
```

ed is the **line** editor, and it needs nothing installed. If you would rather have a full screen, `edit` opens the same record in Microsoft Edit and `micro` opens it in micro — see Programmer commands. The old full-screen editors `sed`, `update.record` and `modify` are all gone; see Not in SD Core.

In `ed`: `i` to insert, type your lines, a full stop on its own line to stop inserting, then `fi` to file and exit.

You do not have to write programs in `ed`. An account's `bp` file is a **directory file** — an ordinary Windows folder with one file per program — so Notepad++, VS Code or any text editor works on it just as well:

```
C:\ProgramData\SD\user_accounts\<account>\bp
```

Save the file, then `basic` and `catalog` it from inside SD as usual. SD folds CR+LF line endings on the way in, so a Windows editor's output needs nothing done to it — see Other hardening.

```
list customers
count customers
```

Commands are lower case now, and so are the VOC records behind them. Typing `LIST` still works — SD tries what you typed, then lower case, then upper. See Lower case.

THIS IS THE POINT AT WHICH MOST THINGS SHOULD FEEL LIKE OpenQM. If anything in ordinary data work behaves differently and is not described in this set, that is worth reporting.

5. Become an administrator

```
logto sdsys
```

You will get a UAC consent prompt unless the session is already elevated. That is the gate — there is no SDSYS password, and there is deliberately no second shared secret held by every administrator.

IF YOU ARE OVER ssh, THIS MAY NOT WORK. A UAC prompt has no interactive desktop there. Start an elevated terminal at the machine instead. See Security.

6. Create an account for somebody else

```
create.account user jane programmer both
```

Three things about that line, and each has caught people out:

<code>programmer</code>	without it, <code>jane</code> gets a standard account and cannot compile, catalogue, edit or create files. See Account types
-------------------------	---

<code>both</code>	one of <code>ssh</code> , <code>api</code> , <code>both</code> , <code>none</code> is required . There is no default
-------------------	---

it needs elevation	creating a Windows account cannot be done from a filtered token
--------------------	---

You will be prompted for Jane's password, masked. **Refusing the prompt creates nothing at all** — a user account cannot exist without a password.

The password is for the API. Console and ssh logins ask for nothing.

7. Sign in as Jane

```
ssh jane@localhost
```

You land directly inside SD, not at a Windows prompt. That is the forced command, and it applies to everyone who connects, administrators included.

Jane cannot log in to Windows at this machine. She is denied the console and Remote Desktop by group membership, deliberately. She reaches SD over ssh or through an API client — which is what the `ssh`/`api`/`both` keyword chose. See ssh access.

`ssh localhost` needs no network and works on a machine with no network connection at all.

8. Leave

`off`

What to try next, in rough order of how likely it is to find something

1. **Your own application data. There is no restore utility**, so the way in is a short BASIC program that reads your exported data and writes the records. Then query it — the query processor is where most of the surface area is.
2. **A client program against the API.** Point it at port 4243 — **not** an ssh tunnel any more. It needs a client library from this release, because the old cleartext login is gone. **The architecture must match the application**, and **The DLL goes in the same directory as the application that loads it** — Windows searches there before `PATH`. **If you have no client of your own, mvDeveloper is free** and is a 32-bit application, so it wants the 32-bit `qmccli1ib.dll`: <https://www.brianleach.co.uk/mvDeveloper>. See API access and Client distribution.
3. **A standard account.** Create one without `programmer` and confirm it can run your application but not edit it.
4. **An upgrade.** Install over the top and check your data survived and `update.accounts` brought the VOC forward.

When something goes wrong

Something SD did, and who did it	<code>audit</code> , in <code>C:\ProgramData\SD\sdsys</code>
Diagnostics, and API connections	<code>errlog</code> , same place
<i>"the account was not created — what happened"</i>	<code>sd-elevate.log</code> , in <code>C:\ProgramData\SD</code>
The installation itself	Check the SD installation on the Start Menu

Other hardening explains which log answers which question — they are not interchangeable.

When you report something, say which build. The release stamp is on the sign-on banner and in `C:\Program Files\SD\changelog`.

Running SD

The service, starting and stopping, and what to do when the last shutdown was not a clean one.

SD is a Windows service and it is already running. Nobody types `sd -start` any more.

Display name	String Database (SD)
Service name	<code>SD</code>
Start type	automatic — Windows starts it at every boot
Created by	the installer, which also starts it
Removed by	the uninstaller

This is a change from the Linux original, where `sd -start` had to be typed after every restart.

Starting and stopping

Stopping the service stops SD and ends every session on the machine, which is exactly what `sd -stop` has always done. Either is fine:

```
Stop-Service SD
Start-Service SD
```

```
sd -stop
sd -start
```

`sd -start` and `sd -stop` tell the truth now

Before this port, both answered from the shared memory segment, which outlives the daemon — so both could report success while doing nothing. Four things changed, and each is a state you may hit while testing.

"SD is already started" IS NOW ONLY SAID WHEN THE DAEMON REALLY IS RUNNING, and it tells you the process id — the Windows one, the number Task Manager and `Stop-Process` use.

IF THE SEGMENT IS THERE BUT THE DAEMON IS NOT — what a killed or crashed SD leaves behind — `sd -start` says so and tells you to run `sd -stop` first. It used to say "*SD is already started*" and do nothing, leaving the system unusable while the command that would fix it reported success.

It does **not** clear the wreckage for you: that would end any sessions still attached to the segment. **The count of those is printed so you can decide.**

`sd -stop` now checks that the daemon actually stopped. A daemon started from an elevated session cannot be stopped from an ordinary one — Windows refuses the signal — and that used to be silent, leaving a daemon running against a segment nothing else could see. You now get a warning naming the process id and the command to stop it with.

Known limit. If the segment has already gone, `sd -stop` has nowhere left to read the daemon's process id from and cannot report on it at all. Check by hand:

```
Get-Process sdwind
```

After an unclean shutdown

If SD is stopped abruptly — the power goes, or the process is killed — it leaves a shared memory segment behind. **On Windows that survives a reboot, where on Linux it would not.**

Earlier builds of this port refused to start on the next boot and said "*Run sd -stop to clear it*", so **the machine came up with SD unavailable to everybody** until somebody logged in and typed it by hand. There was no Linux behaviour to inherit here: on Linux the segment does not survive the reboot at all.

Nothing from before a restart can still be using that segment, so SD now discards it and starts normally, printing:

```
Discarding the shared segment left by the previous boot -  
SD did not shut down cleanly.
```

This changes nothing while the machine is running. A segment belonging to a live SD is still never touched, and `sd -start` still refuses to disturb a system that is already up.

SD will not start a second time inside itself

If you leave SD with `sh` and then type `sd` in that shell, it says so and returns you to the session you already have.

Worth knowing alongside it: `sh` itself needs either an elevated session or a `yes` in `os.users`, so an ordinary account cannot leave SD this way at all, and one reached over ssh never can. See Administrator commands.

The command line

```
sd                enter the SD account named after your Windows login
sd -a             prompt for an account
sd -a<name>       enter account <name> -- refused unless it is your own
sd <command>      run one command -- needs elevation, or batch.jobs
sd -quiet         suppress the displays on entry
sd -u             list current users
sd -k <n> | -k all log out user n, or everybody
sd -start         start the system
sd -stop          stop the system
sd --version      report the version
sd --help         this summary
```

Three of these behave differently from what you may expect.

`sd -a<name>` is refused unless `<name>` is your own account. An administrator no longer opens somebody else's account without ever being in their own — they arrive in their own and reach the rest with `logto`, which is where SD checks whether they are allowed in.

`sd <command>` needs an elevated session, or an entry for that account in `batch.jobs`. Any account can be given one, whatever its tier — standard, programmer or administrator. The account type does not decide what may run; the administrator's list does. That is what makes scheduled jobs possible without handing them administrator rights — see Scheduled jobs.

`sd <command>` runs and exits, and is never asked to set a password. Nor is any session with no terminal — a scheduled task, or a piped script. Only an interactive `sd` with no command after it still asks, and then only of an administrator account that has no password yet.

Earlier builds of this port reached the "needs a password" prompt and blocked for ever on a read that never got input, with nothing in any log because nothing had gone wrong from SD's side.

Where things are

Binaries	C:\Program Files\SD\usr\bin\
The changelog	C:\Program Files\SD\changelog
Configuration	C:\ProgramData\SD\sd.conf
The database	C:\ProgramData\SD\sdsys\
Accounts	C:\ProgramData\SD\user_accounts\, ... \group_accounts\

Audit trail	C:\ProgramData\SD\sdsys\audit
Error log	C:\ProgramData\SD\sdsys\errlog
Elevation helper log	C:\ProgramData\SD\sd-elevate.log

Do not move the binaries. `usr\bin` is load-bearing: shipping `msys-2.0.dll` beside the executable relocates the POSIX root to the DLL's directory minus two components, and only that depth puts `/` on `C:\Program Files\SD\`.

The DLLs ship beside `sd.exe` deliberately — Windows searches the executable's own directory before `PATH`, which avoids Git for Windows's rival `msys-2.0.dll` being picked up. **That failure makes SD report "SD has not been started" while it is running**, which is worth recognising because it looks like nothing else.

The Start Menu

SD	starts <code>sd.exe</code> in the data directory
Check the SD installation	the post-install check, re-runnable at any time. Closes on a keypress

Scheduled jobs

Running an SD command on a timer, and the permit list that decides which ones.

A scheduled task can run an SD command, and only the commands an administrator has named for it.

Typing a command after `sd` — `sd my.report` — needed a session started with *Run as administrator* in earlier builds of this port. A scheduled task does not run that way, so there was no way to have SD do anything on a timer without handing the job administrator rights.

There are two ways past that now:

an elevated session	runs anything, exactly as before
---------------------	----------------------------------

anything else	runs what is listed for its account in the SD system file <code>batch.jobs</code>
---------------	---

Nothing changes for a session you type at. Commands entered at the `:` prompt are unaffected, and so is plain `sd` with no command after it.

There is no password anywhere in this. The job signs in as its own Windows account and SD puts it in the matching SD account, exactly as it would for a person at a keyboard. Nothing has to store a credential for the job to use, and the job grants nobody anything — no other account is in its group.

The permit list

`batch.jobs` is a directory file in `C:\ProgramData\SD\sdsys`, so **one record per account, named after the account**, holding one command name per line.

Only an administrator can change it. It is read-only to SD users, by the same control and the same script as the `os.users` permit list — a user who could add a line to their own record would be granting themselves the command line.

It is keyed by the account you end up in, not by the Windows account name, and the two are usually the same. `sd -a<name>` cannot be used to reach somebody else's list: that form is refused unless the account is your own.

Any account can be given a list, whatever its tier — standard, programmer or administrator. The account type decides nothing here; the administrator's list does.

A standard account cannot write its own paragraph, though. `ed` is not in the standard tier, so the paragraph the job runs has to be put there by somebody who can edit — see Account types.

Both shapes of record are read — one name per line, and a multivalued field 1 — so a record written with `ed` and a record written by a program agree. A name still has to match exactly.

What may be listed

Two rules, and both are tested when the command runs rather than when it is listed:

1. A single name with nothing after it. `my.report` may be listed; `my.report yesterday` may not, and neither will run:

A command run from the command line must be a single name with nothing after it. Put anything the job needs to vary inside the paragraph or sentence being run.

That is the rule doing the security work. "*It must be in the VOC*" would be worth nothing on its own — every verb is a VOC entry — so it is the absence of arguments that stops a listed name being turned into something else.

2. A paragraph or a sentence in that account. A VOC entry of type `PA` or `S`. Anything else — a verb, a file pointer — is refused by name:

MY.REPORT has to be a paragraph or a sentence in this account to be run from the command line, and it is neither.

Setting one up

Four steps, and only the third needs an administrator.

1. Write the work as a paragraph in the account that will run it. A `PA` record in that account's VOC, with the commands on the lines after the type:

```
:ed voc my.report
```

2. Check it runs when you type it, in that account, before putting it on a timer.

3. Add the name to the account's record in `batch.jobs`, from an elevated session:

```
:logto sdsys  
:ed batch.jobs fred
```

One command name per line. Create the record if the account has none.

4. Create the scheduled task. Windows Task Scheduler runs it; two fields carry the whole of it:

Program/script	<code>C:\Program Files\SD\usr\bin\sd.exe</code>
----------------	---

Add arguments `my.report`

The task runs as a Windows account, and that account needs a matching SD account — that is the whole of the sign-in. There is no password to configure anywhere in SD for it.

Do not tick "run with highest privileges". It is not needed, and the whole point of `batch.jobs` is a job that runs without administrator rights. A task that is elevated passes the gate on elevation alone and never consults the list, so it would also be a job nobody had approved a command for.

Use a Windows account you already had — that is the case the design was built around. Accounts that `create.account` makes are denied interactive logon at this machine on purpose, which Task Scheduler may or may not accept as the identity a task runs as.

When it refuses

It fails closed. Every path that does not reach the end of the check leaves the command refused, and each one writes a reason to the audit trail:

WHAT IS WRONG	RECORDED AS
the command line carried an argument	<code>command line carried arguments</code>
<code>batch.jobs</code> is missing or unreadable	<code>batch.jobs could not be opened</code>
the name is not in this account's record	<code>command is not on the account list</code>
the name has no VOC entry at all	<code>no VOC record for the command</code>
it has one, but it is not <code>PA</code> or <code>S</code>	<code>VOC type is not PA or S</code>

The message on the screen for the third of those names both the command and the account, and says where an administrator grants it:

MY.REPORT is not a command that account FRED may run from the command line.

An administrator grants it by adding the name to the account's record in the SD system file `batch.jobs`, which only an administrator can change.

A refusal is not a hang. `sd <command>` no longer walks into a password prompt it cannot answer — see Running SD.

Account types

Standard, Programmer, Administrator, Suspended and Group — what each one may do, and how to make one.

OpenQM gives every account the same VOC and leaves privilege to `SYSTEM` membership. SD Core does not. **An account is created into one of three tiers, and the tier decides what verbs its VOC contains.** There is a fourth tier, **suspended**, which is not a capability at all — it denies entry. Group accounts are a different thing again: a shared place, not a person.

The tier is no longer fixed at creation. `modify.account` moves an account between all four, in either direction, and rebuilds its VOC to match at once. See Changing an account afterwards.

The three capability tiers at a glance

	STANDARD	PROGRAMMER	ADMINISTRATOR
Verbs	81	81 + 42	81 + 42 + 20
VOC records on creation	354	396	416
Can run an application	yes	yes	yes
Can compile, catalogue, edit	no	yes	yes
Can create or configure files	no	yes	yes
Can administer accounts	no	no	yes
Windows <code>Administrators</code>	no	no	yes
Local console and Remote Desktop	denied	denied	allowed

`administrator` **implies** `programmer`. You do not need both keywords.

STANDARD — 82 VERBS

What an application needs and no more: query and list (`select`, `list`, `get.list` and family), spool and print, session and environment (`logto`, `date`, `who`, `set`), screen and message, prompt and input state, and eight read-only inspectors — `search`, `list.diff`, `list.item`, `list.common`, `list.vars`, `report.src`, `report.style`, `format`.

Everything an application built on SD invokes, and nothing that edits code or data in bulk.

PROGRAMMER — 42 MORE

The development set: the compilers, the two full-screen editors, the cataloguer, the file and index definition verbs, the bulk record editors and the process introspection verbs. See Programmer commands for what each one is for.

THE COUNTS ARE ARITHMETIC, NOT OBSERVATION

Installed `NEWVOC` holds 395 names, of which `%t` is a dynamic-file artefact and the two tier lists are never copied — so **392 records reach a full VOC**. `create.account` then adds four of its own (`$command.stack`, `$hold`, `$savedlists`, `bp`):

ADMINISTRATOR	392 + 20 + 4 = 416
PROGRAMMER	392 + 4 = 396
STANDARD	392 - 42 + 4 = 354

A standard account's total did not move when `micro` was added, because `micro` joined `NEWVOC` and `TIER.OMIT.STANDARD` at once — it is on both sides of the subtraction.

If your counts differ, one of the two tier lists differs — which is worth reporting.

The same two numbers are what a tier change reports, so you can predict them: moving between standard and programmer is **42** records either way, and between programmer and administrator **20**. A change that reports a different number, or zero where it should have moved something, is worth reporting for the same reason.

ADMINISTRATOR — 20 MORE

Account and grant administration, system-wide state, and the shell escapes. See Administrator commands.

None of the three is a wall. An administrator can copy any verb into any account's VOC afterwards. **The reduced VOC is the posture an account starts in, not a boundary anything enforces.** The boundaries that are enforced are the operating system's file permissions, the ssh confinement, and the `os.users` permit list — not the contents of a VOC.

Suspended — the fourth tier, and the only one that is a wall

A suspended account cannot be entered. It is for an account that should stop working for a while — somebody on leave, a login being looked into — and it is refused at all three ways in:

ssh, or the console	Account FRED is suspended
<code>logto</code> from another account	Account FRED is suspended
the API	User not allowed in requested account

The API wording is deliberately the same one it gives for an account that does not exist and for one you are not granted, so the API cannot be used to find out which accounts exist or what state they are in.

It takes nothing away, which is why lifting it is free. The VOC is left exactly as it is, no Windows group membership moves, and the tier it displaced is remembered — so bringing the account back puts it exactly where it was, with nothing for you to write down. Suspending is not a substitute for deleting: it is reversible on purpose.

An elevated administrator can still `logto` into a suspended account. That is deliberate — looking at a suspended account is the usual reason to have one, and anybody elevated could lift the suspension anyway. **What a suspension denies is the account's own user.**

And a suspended administrator is still a Windows administrator. SD refuses them; Windows does not. They keep their `Administrators` membership and their `os.users` record, so they can still elevate on the machine and still reach any account they could reach before. **If you are suspending an account to contain somebody rather than to park it, suspend it in Windows too.**

Creating an account

```
create.account user <name> {administrator | programmer}
                        <ssh | api | both | none> {no.query}

create.account group <name> {no.query}

create.account other <name> <pathname> {no.query}
```

ONE OF `SSH`, `API`, `BOTH`, `NONE` IS REQUIRED

There is no default, on purpose. An account that should only ever be reached with `logto` says `none` and means it. The old silent behaviour — ssh yes, API no — could not tell that apart from somebody who had not thought about it.

An administrator account always gets both and needs no keyword. Group accounts take none of this: they have no Windows account.

WHAT CREATING A USER ACCOUNT ACTUALLY DOES

Makes a Windows local account	created disabled, then enabled when the password is set
Creates the group <code>sdu_<name></code>	and writes it to the account record
Joins <code>sdusers</code>	which is what grants access to the data tree
Joins <code>sdsshonly</code>	unless <code>administrator</code> — this is what denies the console and Remote Desktop
Joins <code>Administrators</code>	only with the <code>administrator</code> keyword
Writes the tier to <code>ACCOUNTS</code> field 5	so <code>LOGIN</code> cannot undo it at the next update

Prompts for a password

in SD, masked; it never goes on a command line

A user account cannot be created without a password. Refusing the prompt creates nothing at all. Previously it left an account you could not sign in to.

Elevation is not optional. Creating a Windows account needs an elevated token, and an ordinary SD session has a filtered one. Account creation works from the installer and from an elevated terminal, and not from a normal session.

`create.account user ... ssh` and `... both` are refused when the machine has no ssh server, with a warning saying why: the account would be denied the console and Remote Desktop and have no ssh to arrive on, so it could sign in nowhere. `api` and `none` still work. The test is made against the machine when you type the command, so installing an ssh server later makes `ssh` start working. See Installing SD Core.

Continued in

Managing accounts — group accounts, sharing an account, changing one afterwards, and deleting it.

Managing accounts

Group accounts, sharing one, changing an account afterwards, and deleting it.

This page continues Account types.

Group accounts

A group account is a shared workspace with **no Windows account and no sign-in of its own**. It is how you keep separate work separate, and it is the one kind of account that needs no way in from outside — which makes it the only extra account available on a machine with no ssh server and no API.

```
create.account group payroll
modify.account add payroll fred
```

Reach it with `logto payroll`, or through an F pointer. On a machine with no remote access at all, `logto` into a group account from a session run as administrator.

Sharing a user account

```
grant <account> to <user>
revoke <account> from <user>
list.grants <account>
```

All three need an elevated session, and say so rather than failing obscurely.

Read this twice, because it is the most confusing part of how SD controls access. Entry to an account is membership of the Windows group named in the account's record, and **Windows fixes group membership when you sign in**. So:

- a grant does not reach the person until they sign out of Windows and back in;
- and **somebody you have just revoked keeps the account until they do the same**.

Both verbs print that reminder every time.

`list.grants` also shows the account's own user, which is always there and is not a grant. It is listed anyway so that what you see matches the Windows group it is reporting.

The old field 4 of an `ACCOUNTS` record — a list of accounts allowed in — has been removed, and `list accounts` no longer shows a *Granted to* column. `list.grants` answers that question now.

`list accounts` **shows the tier, and its columns have changed.** The default is `Account`, `Pathname`, `Description`, `Tier`. The Windows group came out to make room: it is always `sdu_` or `sdg_` followed by the account name, so it told you nothing the id did not. Both are still there by name — `list accounts path descr group tier`. **Was is the tier a suspension displaced**, and it is empty on an account that is not suspended.

From an ordinary account the register is reached as `sd.accounts`, not `accounts` — `list sd.accounts`, `ct sd.accounts fred`.

Changing an account afterwards

```
modify.account <name> <standard | programmer | administrator | suspended>
modify.account <name> <ssh | api | both | none>
modify.account <name> <sh-on | sh-off | os-on | os-off>
modify.account add <group> <user>
modify.password {<account>}
```

All of them need an elevated session.

THE TIER

The tier moves in either direction and needs no intermediate step. An account can go programmer, standard, suspended and back to any of them directly. Naming a tier on a suspended account lifts the suspension into that tier — **there is no `resume` keyword**; coming back always says where to.

The VOC is rebuilt there and then, not at the next login, and the verb says what it did:

```
:modify.account fred standard
Account FRED is now STANDARD
VOC: 0 records added, 42 removed, 0 left alone
```

"Left alone" is the count to read. A record is only removed if it is still exactly what SD put there. **Anything you or the account's owner has edited is counted and kept** — it is somebody's work under a verb's name rather than the verb, and a downgrade will not destroy it.

LEAVING `ADMINISTRATOR` TAKES THREE THINGS, AND YOU MUST NAME ONE

Windows `Administrators` membership removed for you

the `os.users` record removed for you

The first two were the account's *because* it was an administrator. ssh and the API are a rule for an administrator and a setting for everybody else, so the command will not guess:

```
:modify.account fred programmer
Say what remote access FRED is to have: ssh, api, both or none
:modify.account fred programmer both
FRED may sign in over ssh and use the API
Account FRED is now PROGRAMMER
```

You cannot suspend your own account, or the one you are standing in. And a group account can be any tier except `administrator` — there is no single person behind it to put in a Windows group.

THE REMOTE ROUTES

The keyword says what the access is, not what to add. So

```
modify.account fred api
```

gives Fred the API **and takes ssh away**. If you want both, say `both`. The message afterwards always names both routes, so you can see what you have left him with.

REACHING THE OPERATING SYSTEM

`sh-on`, `sh-off`, `os-on` and `os-off` set the two fields of the person's `os.users` record without your having to edit it by hand — field 1 is the `sh` verb, field 2 is `OS.EXECUTE` and the two full-screen editors. See Administrator commands.

These four are switches, not names for one state, unlike the tier and the routes above — so `sh-off` leaves `OS.EXECUTE` alone.

WHAT REFUSES ON AN ADMINISTRATOR'S ACCOUNT

All three of ssh, the API and the operating system. An administrator has every route as a rule, and none of them is this verb's to change:

```
:modify.account don os-off
don is an administrator and always reaches the operating system
```

Downgrade the account first if that is really what you want. The tier is the only thing about an administrator that `modify.account` will change.

But the routes are local only, and that is not something you set. An administrator keeps ssh and the API for good, and both stop at this computer: a sign-in from anywhere else is refused, whatever the account record says.

An administrator may not sign in to this machine from another one.

Administration happens at the console, or through a remote desktop or remote-control product installed as a service. Ordinary and programmer accounts are not affected — their routes work from other machines exactly as before.

`set.password` is now `modify.password`. Same verb, same behaviour; the name changed because every account has a password from the moment it is made, so there is nothing to *set* for the first time.

Deleting an account

`delete.account` **asks once, then removes everything.** The single question names exactly what will go, including the Windows account when there is one to remove.

It will not delete a Windows account SD did not create. The question uses shorter wording in that case rather than promising something it will not do.

Two things that catch people out

1. `update.accounts` **never takes a verb away.** SD only ever *adds* records to a VOC at an update. An account created before a verb was withdrawn keeps it, and running `update.accounts` will not remove it.

`modify.account <tier>` **is the one that does remove them**, and it is the only thing that ever will. If you need a verb gone from an account, change the tier; `update.accounts` will not undo it afterwards either, because the tier is recorded and every update applies it.

2. `sdusers` **membership needs a fresh logon.** Same reason as grants. After being added to the group, sign out of Windows and back in, or you cannot read the data tree at all — and the symptom looks like a broken install rather than a permissions problem.

Administrator commands

The 23 verbs an administrator account gets above a programmer, and how to use them.

An administrator account receives everything a programmer account does, plus the 23 verbs below. In OpenQM most of these lived in `SYSTEM` and you reached them by being there; in SD Core they are in the administrator's **own** account VOC, so an SD administrator no longer has to `logto sdsys` to administer anything.

Being an SD administrator means being an elevated Windows administrator. There is no SD password that confers it. Several of these verbs check for elevation and say so rather than failing obscurely — see Security.

Accounts

```
create.account user <name> {administrator | programmer} <ssh|api|both|none>
create.account group <name>
delete.account <name>
modify.account <name> <standard | programmer | administrator | suspended>
modify.account <name> <ssh|api|both|none>
modify.account <name> <sh-on | sh-off | os-on | os-off>
modify.account add <group> <user>
update.accounts {all}
clean.account
```

Covered in full in Account types. The points worth repeating here:

- **One of `ssh`, `api`, `both`, `none` is required** on `create.account user`. There is no default.
- **`modify.account` says what the access is, not what to add.** `modify.account fred api` takes ssh away. The tier keyword works the same way.
- **The tier can be changed after creation**, in either direction, and the VOC is rebuilt at once. `suspended` is a fourth tier that denies entry.
- **`update.accounts` only ever adds VOC records**, never removes them — `modify.account <tier>` is the only thing that removes one.
- **Nothing here changes an administrator's access.** ssh, the API and the operating system are all a rule for that tier; downgrade the account first.

`clean.account` tidies an account's workspace. `update.accounts` is the one you run in each account after upgrading SD.

Grants

```
grant      <account> to   <user>
revoke     <account> from <user>
list.grants <account>
```

All three need an elevated session.

A grant or a revoke does not take effect until the person signs out of Windows and back in. Entry to an account is Windows group membership, and Windows fixes that at logon. **Somebody you have just revoked keeps the account until they get a new token.** Both verbs print the reminder every time.

Passwords

```
modify.password {<account>}
```

`set.password` **was renamed to** `modify.password` — every account now has a password from the moment it is created, so there is nothing to *set* for the first time.

A password cannot be typed on the command line. `modify.password` refuses one given as an argument. A password on a command line is visible to any local user through Task Manager or `Get-CimInstance Win32_Process`, so SD prompts for it instead, masked.

The password matters for **API logins only**. Console and ssh logins ask for nothing — see Security.

Locks and sessions

```
lock / unlock
list.locks / clear.locks
listu / list.readu
logout
```

`unlock` **is the one to know about.** It clears a record lock left behind by a session that died holding one. Without it the only way to release such a lock is to stop and restart SD, which disconnects everybody.

An earlier changelog entry claimed `unlock` had never worked on any release this port was built from, because its VOC entry carried description text in the type field. **That conclusion was wrong and has been withdrawn.** SD allows a type code followed immediately by comment text — the same rule `PI`, `PI/open` and `UniVerse` follow — so `Verb to unlock records` reads as type `v` with a comment, and `unlock` dispatched normally all along. The record has been rewritten as a bare `v` for consistency, and behaves no differently. `cotyp` carried the same style of entry and was likewise working.

`listu` and `list.readu` report sessions and read locks; `logout` ends another session.

System state

```
config
set.date
```

`config` reports the configuration parameters in force.

`CREATUSR` is gone. `config` no longer lists it and `config('CREATUSR')` returns nothing. A `CREATUSR` line in `sd.conf` is still accepted and ignored, so a configuration file brought from a Linux install still works. You can delete the line.

The shell escapes — `sh` and `!`

Who may use `sh` is a list you keep, not a matter of elevation.

This is one of the larger behavioural changes in the port. In earlier builds of it `sh` required an elevated session — and **an ssh session can never be elevated**, so programmers, the people who actually need a shell, were the ones who could never have one.

THE LIST

One record per person, **keyed by the name they sign in with, not by the account name**. The login name follows the person, so it does not change when they `logto` somewhere else.

HOW YOU GRANT IT

Four keywords on `modify.account` do it without your editing anything:

```
modify.account fred sh-on      the sh verb and ! at the prompt
modify.account fred sh-off
```

```
modify.account fred os-on      OS.EXECUTE, and the edit and micro editors
modify.account fred os-off
```

`create.account` takes the two `on` forms. **They are four switches over two fields**, not four names for one state, so `sh-off` leaves `OS.EXECUTE` alone — and the verb prints the resulting record, both fields, every time.

An administrator-tier account gets both fields without being asked, including the account the installer makes for whoever installs SD. That is why an administrator reaches the editors and the shell from an ordinary, unelevated session.

And the four keywords refuse an administrator, in both directions:

```
:modify.account don os-off
don is an administrator and always reaches the operating system
```

An administrator has all three routes — ssh, the API and the operating system — as a rule. Downgrade the account first if that is really what you want.

All three are local only. The grant cannot be taken away and it does not reach past this computer: an administrator signing in from another machine is refused, on ssh and on the API alike, after the password has been checked.

An administrator may not sign in to this machine from another one.

This is not a setting and there is no keyword for it. Administration happens at the console, or through a remote desktop or remote-control product installed as a service. Ordinary and programmer accounts reach the machine remotely as before.

OR EDIT THE RECORD BY HAND

The record is ordinary data. From an **elevated** session, because the file is writable only by an administrator:

```
logto sdsys
ed os.users don
```

The record is two lines and nothing else. To give `don` the shell and the editors:

```
yes
yes
```

In `ed`: `i` to insert, type the two lines, a full stop on its own line to stop inserting, then `fi` to file and exit. **Field 1 is `SH`, field 2 is `OS.EX`** — the order matters and there is nothing else in the record.

WHAT YOU WANT <code>DON</code> TO HAVE	FIELD 1	FIELD 2
nothing outside SD	<i>no record at all</i>	
the editors, but no shell at the prompt	anything but <code>yes</code>	<code>yes</code>

WHAT YOU WANT DON TO HAVE	FIELD 1	FIELD 2
a shell, but programs may not shell out	yes	anything but yes
both	yes	yes

is the **Windows login name, not the SD account name**. They are usually the same; where they are not, this file wants the one the person signs in with.

The change takes effect on their next command, not at their next login — the list is read when the shell or the editor is asked for.

To take it away, set the field to anything else or delete the record. Deleting the record removes both.

FIELD	CONTROLS	VALUE
1	SH — a shell at the command prompt	yes , or anything else for no
2	OS.EX — OS.EXECUTE from inside a program, and the edit and micro editors	yes , or anything else for no

Field 2 is what lets a programmer use the full-screen editors. They run an editor outside SD, so they are reaching the operating system whatever the VOC tier says. A programmer with the verb and no in field 2 is told the command is not available and what to ask for — see Programmer commands.

A missing record, or a missing file, means no. An installation that has never set up denies both to ordinary accounts.

One case Hand-editing is the only way out of. An account promoted to administrator gets both fields written; an account that was *adopted* over a Windows login which already had a record saying **keeps that record**, and because the four keywords refuse an administrator, no verb can then change it. is the way out. It is rare, and it is the reason this section still documents the file rather than only the keywords.

BOTH FIELDS ARE ENFORCED

used to be unchecked entirely, so **any account that could write a program had the operating system** and the restriction could be walked around by anyone able to type BASIC. Field 2 is now what decides whether may run. The refusal reads *is not permitted to use OS.EXECUTE*.

The two fields are independent, and the useful combination is the third row:

unlisted	sh refused	OS.EXECUTE refused	
SH=yes OS.EX=no	sh RUNS	OS.EXECUTE refused	
SH=no OS.EX=yes	sh refused	OS.EXECUTE RUNS	<- programs may shell out; the person at the prompt may not
elevated	sh runs	OS.EXECUTE RUNS	

SD's own system programs are exempt, so `create.account` and the `sh` verb work as before.

THREE RULES THAT GO WITH IT

Only an administrator can edit the list. `os.users` is read-only to everybody else on disk, and **that ACL is the whole of what stops a user granting themselves a shell. Do not loosen those permissions.**

A listed person gets a real shell — pipes, redirection and chaining all work: `sh dir | more`. Anyone not on the list who reaches `sh` by being elevated still gets the restricted form, which rejects those characters.

An elevated session keeps `sh` whatever the list says, deliberately, so an empty list cannot lock the machine's own administrator out.

NEITHER IS AVAILABLE OVER THE API

`sh` and `OS.EXECUTE` are refused to a session that arrived over the API. **Before this port they were not**, and on Windows that was worse than it sounds: in earlier builds of this port they ran **as the LocalSystem account**, so a remote client could run any command on the machine with full privilege — more than the administrator at the keyboard gets. An API session is no longer treated as an administrator for any purpose.

The full list of the 20

`create.account` · `delete.account` · `modify.account` · `update.accounts` · `clean.account` · `grant` · `revoke` ·
`list.grants` · `unlock` · `modify.password` · `config` · `listu` · `list.readu` · `list.locks` · `clear.locks` · `lock` · `logout`
· `set.date` · `sh` · `!`

Twelve of these left NEWVOC on 24 Aug 2026 — `config`, `listu`, `list.readu`, `list.locks`, `clear.locks`, `lock`, `logout`, `set.date`, `sh`, `!`, `clean.account` and `umask`. They are no longer copied into any account's VOC; administrators receive them from `VOC_TEMPLATE` instead. **A programmer or standard account created before that update keeps whichever of them it had**, because `update.accounts` never removes a record.

`umask` **was removed from every tier**. It controls POSIX file-mode bits, which Windows does not use for security — all file access runs through the ACLs an SD account already receives. Setting a umask on Windows produces no security effect and would only be misleading. The record types stay compiled, but no VOC in any tier dispatches to them.

Programmer commands

The 42 development verbs a standard account does not get, and what withholding them does and does not do.

A **standard** account gets 82 verbs — enough to run an application and nothing that edits code or data in bulk. A **programmer** account gets those plus the 42 below. An **administrator** account gets all of them, plus 23 more.

This split does not exist in OpenQM, where every account gets the same VOC. None of the verbs below is new; what is new is that an account has to be created as `programmer` or `administrator` to receive them.

```
create.account user fred programmer both
```

What is withheld, by function

COMPILE, CATALOGUE AND RUN

<code>basic</code>	compile SD BASIC source
<code>catalog</code> · <code>catalogue</code>	add to the catalogue
<code>delete.catalog</code> · <code>delete.catalogue</code>	remove from it
<code>compile.dict</code>	compile dictionary items
<code>run</code>	run a compiled program
<code>map</code>	show a program's map
<code>generate</code>	generate source
<code>phantom</code>	start a background process

Cataloguing globally requires an administrator. Adding to or removing from the system-wide catalogue is an administrator action; private and local cataloguing still work in a programmer's own account. That is a separate control from the VOC tier.

EDIT AND DEBUG

<code>ed</code>	the line editor. Needs nothing installed
<code>edit</code>	a full-screen editor — opens the record in Microsoft Edit
<code>micro</code>	a full-screen editor — opens the record in micro
<code>debug</code>	the BASIC debugger
<code>pstat</code> · <code>pdebug</code> · <code>pdump</code> · <code>dump</code>	process introspection

The debug family moved from administrator to programmer deliberately — a programmer needs these to debug their own code.

EDITORS

There are two, and they behave identically. The verb chooses the editor and nothing else changes:

<code>edit</code>	Microsoft Edit — ships in current Windows builds
<code>micro</code>	micro — never ships with Windows; the installer fetches it

```
edit bp myprog
micro bp myprog
edit dict customers name
```

`edit` **used to be a second name for** `ed`, **and is not any more.** If you have been typing it to get the line editor, you will now get a full screen.

Either verb writes the record to a working copy, opens the editor on it, reads it back, and asks whether to save. For a `bp` record it then offers the compile and the catalogue, which is the loop the old `micro` verb had.

Both are terminal editors, so both work over ssh as well as at the console. Where a machine does not have one, the SD installer installs it; if that could not be done, the verb says so and names the command that installs it.

`C:\ProgramData\SD\install-editors.log` records what the installer found.

Only `micro` **highlights SD basic.** Microsoft Edit has no syntax highlighting at all, which is the one real difference between the two verbs:

<code>micro</code>	statements, reserved words, intrinsic functions, <code>@variables</code> , <code>\$directives</code> , labels, strings, numbers and comments
<code>edit</code>	plain text

It applies to a `bp` record and to nothing else. SD names the working copy so that micro can recognise the language — a record edited out of any other file is treated as plain text, which is correct for a VOC entry or a data record.

The word lists are generated from the compiler. They come out of `BCOMP`'s own tables — **218 statements, 37 reserved words and 176 intrinsic functions** — so the highlighting cannot drift from the language. **If a name you expect is not coloured, that is worth reporting:** it means the two have come apart, which is exactly what generating them was meant to prevent.

Nothing is installed into your profile. SD ships the rules with itself and points micro at them, so they work for every account on the machine including the ones that cannot log in to Windows.

`ed` is unaffected and is still there. Nothing has been taken away.

WHAT THE EDITORS ARE GOOD FOR, AND WHAT THEY ARE NOT

They are text editors, so they suit a record whose content is lines of text:

BASIC source in a <code>bp</code> file	what they are for
VOC records	fine — a VOC record is a few short fields
Dictionary records	fine for a simple one; see the limit below
Data records with multivalues	fine — see the tokens below
Data records with subvalues	fine — see the tokens below

A field is a line and that part needs no explanation. SD writes the working copy with one field per line, so moving between fields is moving between lines.

A value mark is not a line, and neither is a subvalue mark. Both are control characters an editor cannot show, so each has a token you can type:

TYPE	TO GET
<code>~~</code>	a value mark
<code>~`</code>	a subvalue mark

SD converts marks to tokens on the way into the editor and tokens back to marks on the way out, so multivalues and subvalues are both ordinary text while you are editing.

SMITH~~JONES~~BROWN

is a three-value field, and

```
RED~`BLUE~~GREEN
```

is two values, the first of which has two subvalues.

A record that cannot be written this way is refused, not mangled. Some records would come back different from how they went in — one that already contains `~~` as data, for instance, or one with a `~` sitting immediately before a mark, where the tilde and the token run together. Before opening the editor, SD converts the record and converts it back; **if the result is not what it started with, the verb refuses and names `ed`**, which needs none of this.

Text marks are not converted, and are covered by the same refusal rather than being left to surprise you.

A compiled dictionary record is truncated to its first 15 fields while you edit it, and recompiled with `cd` when you save — the same thing the old `micro` verb did.

GIVE THESE VERBS ONLY TO PEOPLE YOU TRUST

An editor can write anywhere its user can write. It opens the record you named, but nothing stops the person then opening any other file on the machine that their Windows account may open — inside the SD data tree or outside it altogether. **That is not a hole in SD; it is what an editor is**, and it is the reason these verbs are behind `OS.EXECUTE` permission and not merely behind the programmer tier.

So `os.users` field 2 is a statement of trust in a person, not a convenience. Before granting it, ask the same question you would ask before giving somebody the shell — because in terms of what they can reach on disk, you are.

Neither editor can run a command, so neither is a shell. **What they are is read and write access to the filesystem, with the account's own Windows permissions.** See Security.

BOTH EDITORS NEED `OS.EXECUTE` PERMISSION AS WELL AS THE VERB

Having the verb is not enough. An editor runs outside SD, so reaching one is reaching the operating system — and who may do that is **field 2 of your record in `os.users`**, the same field that governs `OS.EXECUTE` from inside a program. Two gates, and both have to pass:

the VOC tier	decides who has the verb — programmer and administrator
--------------	--

<code>os.users</code> field 2	decides whether it runs
-------------------------------	--------------------------------

An elevated session passes on its own, exactly as `sh` does, so an empty list cannot lock the machine's own administrator out.

A missing record, or a missing file, means no — the same direction `sh` fails in.

If you have the verb and not the permission you get told so by name, and told what to ask for:

```
edit is not available to fred.  
It runs an editor outside SD, so it needs OS.EXECUTE permission: field 2  
of your record in the SD system file os.users, which only an administrator
```

can change.
ed, the line editor, needs none of this.

An administrator grants it — see Administrator commands.

A session with no terminal is refused first and separately: an API session or a piped script has nowhere to draw a full screen, and is told that rather than being told about `os.users`.

WHAT AN EDITOR CAN REACH, and it is worth knowing before you grant it. An editor can open any file the person running it is allowed to open, so both verbs reach beyond SD's own files. Neither is a shell — neither editor can run a command. That is what field 2 is deciding, and it is why the verb alone was not made enough. See Security.

OVER SSH

A terminal editor is the point of an ssh session. An ssh session reaches SD through a terminal like any other, and SD hands the editor that terminal rather than reading it through a pipe.

AN ssh SESSION CAN NEVER BE ELEVATED, though, so an administrator arriving that way needs an `os.users` entry the same as anybody else — being an administrator is not enough on its own over ssh.

If an editor misbehaves over ssh and not at the console, that is worth reporting with the terminal you connected from.

The removed full-screen editors are a different matter: `sed`, `update.record` and `modify` are gone and are not coming back. See Not in SD Core.

FILES

<code>create.file</code> · <code>delete.file</code> · <code>clear.file</code>	the life of a file
<code>configure.file</code>	change a file's configuration
<code>analyse.file</code> · <code>analyze.file</code>	report on a file's internals
<code>fstat</code>	file statistics
<code>hsm</code>	hashed-file statistics monitoring
<code>set.trigger</code>	attach a trigger
<code>cd</code>	change directory

INDEXES

`create.index` · `delete.index` · `build.index` · `make.index` · `list.index`

<code>copy</code> · <code>copyp</code>	copy records
<code>delete</code>	delete records
<code>rename</code>	rename records
<code>reformat</code> · <code>sreformat</code>	reformat
<code>sort.item</code>	sort
<code>cname</code>	change a record's name
<code>delete.common</code>	clear a common block

What the split does and does not do

It is a posture, not a boundary. An administrator can copy any verb into any account's VOC afterwards. A reduced VOC is where an account **starts**, not something SD enforces at run time.

What actually is enforced is elsewhere, and worth knowing before you rely on the tier for anything:

File permissions	Windows ACLs on the data tree — see Security
Where an account may sign in	the <code>sdsshonly</code> deny rights — see ssh access
Reaching the operating system	the <code>os.users</code> permit list, both <code>sh</code> and <code>OS.EXECUTE</code>
What an API session may open	the containment gate, rooted at the account the session stands in

A programmer with `OS.EX` set has the operating system, whatever their VOC says, because they can write and run BASIC. That is why field 2 of `os.users` exists and why it is now enforced — see Administrator commands.

Two things to know when you compile

`basic` **no longer creates an object file it can never open again.** Compiling into a reused file name previously produced an object SD could not subsequently open.

Object code and the catalogue are replaced on upgrade. The compiled programs, the BASIC source SD ships, the messages, include records and VOC templates are all overwritten by a new release. **Anything you have written into the SDSYS `bp` file, and anything you have compiled from it, survives an upgrade untouched** — SD now ships nothing into that file at all, so it is created empty and is yours.

The full list of the 42

basic · catalog · catalogue · delete.catalog · delete.catalogue · compile.dict · cd · generate · phantom · run ·
map · debug · ed · edit · micro · create.file · delete.file · clear.file · configure.file · analyse.file ·
analyze.file · fstat · hsm · set.trigger · create.index · delete.index · build.index · make.index · list.index ·
copy · copyp · delete · rename · reformat · sreformat · sort.item · delete.common · cname · pstat · pdebug ·
pdump · dump

This list was 42, then 41 when `modify` was removed from SD Core, and is 42 again now `micro` is back. A standard account's count did not move on either occasion: both verbs were on both sides of the arithmetic at once — in `NEWVOC` and in the standard tier's omit list.

ssh access

How people reach SD on this machine, why it is ssh, and the one thing it costs you.

Accounts SD creates cannot log in to Windows at this machine. They are denied the physical console and Remote Desktop, deliberately and by group membership. Local terminal access belongs to administrators, who have ordinary Windows accounts.

They reach SD over ssh, or through an API client, or both. Which of those an account may use is chosen when it is created and is a required keyword — see Per-account control below. This page covers the ssh route; API access covers the other.

This has no equivalent in OpenQM or in SD on Linux, and it shapes almost everything else about running SD Core on Windows.

The rule, and the one exception

	CREATE.ACCOUNT USER X	CREATE.ACCOUNT USER X ADMINISTRATOR
Windows group	standard user	Administrators
Administers SD	no	yes
Local console / Remote Desktop	denied	allowed
ssh	if granted	yes
API	if granted	yes

"Denied the console" is about logging in to Windows, not about reaching sd. An account with `api` and no `ssh` is a perfectly normal thing to create — a person using a custom GUI client that talks to SD does not need a terminal session at all.

Multi-user access over remote Desktop is not supported. This is not an oversight and not a gap to be filled later. One Windows setting covers Remote Desktop and the physical keyboard together, so an account allowed to use RDP could also walk up to the machine and log in — which was never the intention. A verb that lifted the restriction (`RDPACCOUNT`) was built and deleted the next day for exactly that reason.

The rule that now holds without exception: nobody SD creates can log in to Windows at this machine unless they are already a Windows administrator. **SD accounts reach the machine over ssh or through an API client.**

If you want multi-user Remote Desktop, you want Windows Server and RDP client access licences, and probably a commercial product built for it.

How it is done

Two Windows user rights, applied **to a group, once** — not per account:

SeDenyInteractiveLogonRight	blocks the physical console
SeDenyRemoteInteractiveLogonRight	blocks Remote Desktop

Both are on the group `sdsshonly`, and `create.account` joins every non-administrator account to it.

Network logon is not denied, and must not be. Win32-OpenSSH authenticates with a network logon — cleartext network logon for passwords, S4U for public keys — so denying it would lock out the very access this exists to preserve. **This is the trap in the design and it is the one thing to get right.**

It cannot be `sdusers`. That group grants access to the data files and administrators are in it too, so denying console logon there would lock administrators out of their own console. The two groups answer different questions and must stay separate.

An ssh session lands inside SD

`sshd_config` carries a global `ForceCommand`, so signing in over ssh puts you straight into SD rather than at a Windows prompt.

That applies to everyone who may connect, administrators included — the rule is about the route in, not about who took it.

THE COST: SCP AND SFTP STOP WORKING INBOUND

NO FILE CAN BE PUSHED TO THIS MACHINE OVER ssh, BY ANYBODY, ADMINISTRATORS INCLUDED. The command is forced, so there is no file-transfer subsystem left to run. This is deliberate and is the accepted cost of the above.

The cost is INBOUND ONLY, and that is the whole of the answer. `ForceCommand` applies to sessions where this machine is the ssh *server*. WinSCP or `scp` running **on** this machine, connecting outward, makes it the *client*, and `sshd_config` is not consulted at all.

So copy files by Pulling them, not pushing them. Sit at the machine or connect with Remote Desktop — both untouched, because administrators are never put in `sdsshonly` — and fetch what you need from there. Outbound connections are not firewalled.

Remote-control tools that copy files are unaffected, and so are the console and Remote Desktop.

Do not "fix" this with a `Match Group administrators` exemption. Beyond giving administrators a PowerShell prompt instead of SD — which is *more* access than the global form gives them — it may not even work: `sshd_config` takes the **first** obtained value for a keyword, and SD's block is inserted before the first `Match`, so a later `ForceCommand none` is not guaranteed to override the earlier global one. Pulling avoids the question entirely.

Who may ssh in at all

`AllowGroups` in `sshd_config` is the second layer: the deny rights stop local login, `AllowGroups` decides who may connect.

THE "Limit ssh" CHECKBOX IS GONE. IT IS NOW A STATEMENT. SD limits ssh to SD users and administrators and puts every ssh session straight into SD. It had been ticked by default for some time and was not really meant to be turned off, so presenting it as a checkbox suggested a choice that was not one. It is described on the *Before you install* page instead, with the scp cost stated plainly.

The reason it can no longer be declined is that the machine it existed for — one with somebody else's ssh server — is now turned away before the install starts.

Your existing `sshd_config` is kept as `sshd_config.before-sd`, and uninstalling SD puts it back.

IF SD DID NOT INSTALL YOUR SSH SERVER

SD never reconfigures or restarts an ssh server it did not install — it may be managed by policy. On such a machine, apply the confinement yourself, once, from an elevated prompt:

```
powershell -File "C:\Program Files\SD\allow-ssh-groups.ps1" -Installed
```

It checks the result with `sshd -T` and puts the original back if anything is wrong, keeping a copy as `sshd_config.before-sd`.

It also **refuses outright** if `sshd_config` already carries an `AllowGroups`, `AllowUsers`, `DenyGroups` or `DenyUsers` line — that is somebody's policy and it is left alone.

Reaching the machine from the network

Port 22 is closed to other computers unless you tick the box during installation. The rule is scoped to `127.0.0.1,::1` otherwise.

IF YOU INSTALLED A BUILD BEFORE 25 Aug 2026, CHECK THIS. The narrowing step failed **every single time it ran**, leaving port 22 open to the local network even when the box was unticked. The installer's closing page said so: *"Setting who may reach ssh FAILED, and Windows' own default is in force — port 22 open to your local network."* The recovery command it offered failed for the same reason. Only machines where SD installed ssh itself are affected. Fix it with:

```
powershell -File "C:\Program Files\SD\ssh-firewall.ps1" -Installed -Restrict
```

It now reports *"ssh is reachable FROM THIS MACHINE ONLY"*. Check the current setting with `-Show`, or look at the rule **OpenSSH SSH Server (sshd)** in `wf.msc`.

Connecting to SD on your own machine is unaffected, over IPv4 and IPv6 alike — Windows does not filter traffic that never leaves the machine, so `ssh localhost` keeps working whichever way the setting is left. **A local-only installation is served entirely by `ssh localhost`.**

Per-account control

An account is told at creation which routes it may use, and one of the four keywords is required:

REACHES SD BY	
<code>create.account user fred ssh</code>	a terminal session over ssh
<code>create.account user fred api</code>	an API client only — no terminal, and nothing on this page applies to them
<code>create.account user fred both</code>	either
<code>create.account user fred none</code>	neither — reachable only with <code>logto</code> from another session

An `api` account never touches any of this. No ssh session, no `ForceCommand`, no port 22. For people running a GUI client against SD, that is the account shape they want, and the ssh configuration on this page does not apply to them.

`modify.account fred both` changes it afterwards — and remember the keyword says what the access **is**, not what to add, so `modify.account fred api` takes ssh away. **Administrators always have both, and it cannot be taken from them — but for an administrator both mean *from this machine only*.** See Account types, and the section on remote administration below.

A suspended account is refused after the connection is made, not before. `modify.account fred suspended` does not touch the `sdssh` group, so sshd still accepts Fred's connection and SD still starts — and then refuses him:

```
Account FRED is suspended
```

A suspension is an SD control, not a network one. That is what makes lifting it free: nothing was withdrawn, so nothing has to be restored. If you need the connection itself refused, take the account out of `sdssh` too with `modify.account fred none` — and remember you must then put it back by hand.

What ssh-only does not mean

The deny rights control *where* an account may log in, not *what it may run*. Confining a user to SD rather than to a shell is the separate `ForceCommand` control above, and reaching the operating system from inside SD is the separate `os.users` permit list — see Administrator commands.

And ssh-only does not give users isolation from each other's data. Every SD process opens the database under the invoking user's own token, so everyone who uses SD needs file access to the tree and can read another account's directory from outside SD. See Security.

An administrator cannot sign in from another machine

If your SD account is an ADMINISTRATOR account, SD refuses an ssh connection from another computer. You will see:

```
An administrator may not sign in to this machine from another one.
```

and the connection ends. The same applies to the API.

ssh and the API still work for an administrator on this machine. A loopback connection — `ssh you@localhost`, or an API client running on the same box — is admitted normally. It is *remote* that is refused, not ssh.

Nothing changes for ordinary accounts. Standard and programmer accounts keep ssh from anywhere, exactly as before, and so does a Windows administrator whose SD account is an ordinary one. **It is the SD account tier that decides, not Windows group membership.**

WHERE ADMINISTRATION HAPPENS

- at this machine's own console, or
- through a remote desktop or remote-control product **installed as a service.**

A per-user install of a remote-control tool is not enough: it cannot display the Windows consent prompt, and the operator sees a frozen screen instead.

WHY

Administration needs a screen Windows can draw its consent prompt on. That prompt is what makes an elevation something a person agreed to, rather than something that merely happened. A connection from another machine has no such screen.

And the token an ssh session carries is not a filtered one. OpenSSH runs as a system service and builds the sign-in token itself, so the filtering that applies to an ordinary local sign-in never applies to it. Without this refusal a Windows administrator arriving over ssh would hold **full** administrator rights, with nobody asked to consent to anything. That is the hole this refusal closes.

IF YOU RELY ON REMOTE ADMINISTRATION TODAY

It will stop working when you upgrade, and that is deliberate. Use the console, or a service-installed remote desktop. If the account only needs ordinary, non-administrative work from another machine, ask an administrator to change its tier instead.

Scheduled tasks are affected too. A task that runs unattended has no screen either, so one signing in as an administrator account is refused. Give it an ordinary account and list the command it runs in the SD system file `batch.jobs`.

API access

The client API, the port it answers on, the login that replaced the old one, and what a remote session may reach.

The client API is the reason this port exists. SD Core for Windows is built to be used as a back end data store reached through the API, and that is the tie-breaker on most other design questions here.

The API is a normal way for any account to use SD, not a facility reserved for developers and administrators. A person running a custom GUI program that talks to SD needs API access and may need nothing else — no ssh, no terminal, no development verbs. A standard-tier account with `api` access is an ordinary thing to create, and probably the commonest shape a deployed system will have:

```
create.account user jane api
```

Your application code does not change. `SDConnect()` and `SDConnectLocal()` take the same arguments and return the same things. **What changed is underneath: the login protocol, the port, the identity a session runs as, and what it is allowed to open.**

The account's tier and its access route are independent. Tier decides which verbs are in the VOC (Account types); `ssh` / `api` / `both` / `none` decides how the account is reached. A standard account with API access, a programmer with ssh only, and an administrator with both are all normal.

The login is SCRAM-SHA-256, and the old one is gone

A client that sends a password in clear is refused. It gets *"Cleartext login is no longer supported; this server requires SCRAM authentication"* and the connection drops.

The API used to send the user name and password as plain text — anything able to watch the connection could read the password. It now runs a challenge and response: the server sets a puzzle only someone who knows the password can answer, and **the password itself is never sent in any form.**

The server also proves itself to you. It finishes by returning a value only the real server can compute, and the client refuses the connection if it does not match. Another program that grabbed the port before SD started cannot pretend to be SD in order to collect passwords.

WHAT YOU HAVE TO DO

Use a client library from this release or later.

An older one is refused outright

Run `modify.password` again for every account that uses the API. The stored credentials changed shape and the old ones cannot be converted

An account whose password has not been re-set is refused, and the refusal reads as a wrong password. From the server's point of view there is no credential to check. If a working client suddenly cannot log in after an upgrade, this is the first thing to try.

The old credentials cannot be converted because **the password was never kept anywhere, by design** — there is nothing to convert them from.

Programs using the `!sdclient` class are covered. The class module BASIC programs use to reach another SD server speaks the new login too. Your code does not change — `connect()` takes the same arguments — but the program has to be running under this release **at both ends**.

`SDConnectLocal()` **connections are unaffected.** They send no password and never did.

The port

`sd.conf` sets `APIPORT=4243`, and the server accepts API connections on any network interface. **Earlier builds of this port shipped it commented out and listened on `127.0.0.1` only**, which is why an ssh tunnel was needed.

If you tunnel, stop. `ssh -L 4243:127.0.0.1:4243 user@host` still works but is no longer what the design expects, and it is not tested. Point the client straight at port 4243 on the server.

Reaching the port from another computer is off unless you tick the box during installation. To change it afterwards, from an elevated prompt:

```
powershell -File "C:\Program Files\SD\api-firewall.ps1" -Open
powershell -File "C:\Program Files\SD\api-firewall.ps1" -Restrict
```

To turn the API off altogether, comment out the `APIPORT` line in `C:\ProgramData\SD\sd.conf` and restart SD. With no `APIPORT` set, SD creates no socket at all — "no API" is a real state, not just a firewall rule.

Declining the API at install time leaves no listener at all. The installer writes an `sd.conf` with no `APIPORT` line, so nothing listens and there is no firewall rule to open. The API box is unticked by default. `remote.api on` puts a listener back.

`APILOGIN` **is not an off switch.** It decides whether the API demands a password. `APILOGIN=0` is the **weaker** setting, not the safer one. Do not reach for it.

Reaching the port is not getting in

A caller must clear three gates, in this order:

1. **Complete the SCRAM exchange** against a password held for that account — so **an account with no password cannot connect at all**.
2. **Be a member of the `sdapi` group**, which an account joins only when you give it API access. `create.account user fred api` or `... both` does that; `ssh` or `none` does not.
3. **If the account is an administrator, be connecting from this machine**. An administrator is refused an API session from any other computer, however good the password. Ordinary and programmer accounts are not affected.
4. **Pass the account's own group check**.

Gate 3 catches only the administrator tier, and it is not a setting — there is no keyword that turns it off. An administrator always has API access and it always stops at this computer:

An administrator may not sign in to this machine from another one.

A local API client is unaffected, which is the point: an application on the server connecting to `127.0.0.1` works as an administrator exactly as before. The reason the tier is treated this way is that an administrator's session can reach the operating system, so SD requires one Windows can show a consent prompt on — the console, or a remote desktop or remote-control product installed as a service. The Administrator set covers it in full.

Failed API logins are written to the audit trail, with the reason — an administrator refused this way is recorded with the address it came from. See Other hardening.

A session is confined to its own account

UNTIL 21 Aug 2026 A CLIENT CONNECTING OVER THE API COULD OPEN ANY FILE ON THIS MACHINE — INCLUDING THE FILE SD KEEPS PASSWORDS IN. Holding one ordinary account's password was enough, and no administration command was needed. **If you have used a build older than that, treat the passwords of every account as having been reachable.**

Still allowed	everything inside its own account, and the shipped SDSYS files every account needs — messages, <code>syscom</code> , the dictionaries, <code>sd.voclib</code> . Ordinary programs are unaffected
----------------------	--

No longer allowed	opening, renaming, deleting or listing anything else
--------------------------	--

A refused `OPEN` takes the `ELSE` branch and `STATUS()` is 3035, which means *not permitted* rather than *not found*. That distinction matters when you are debugging: 3035 is a containment refusal, not a missing file.

A suspended account is refused here too, and deliberately says nothing about why. `modify.account fred suspended` denies the API as well as ssh, and the message is the one this page already gives for an account that does not exist and for one you are not granted:

User not allowed in requested account

All three answer identically on purpose, so the API cannot be used to enumerate which accounts exist or what state they are in. If you are debugging a client that has suddenly stopped connecting, `list accounts` from an SD session is where the answer is — the `Tier` column will read `SUSPENDED`.

IF YOUR DATA LIVES OUTSIDE AN ACCOUNT

Name the directory in the `NETDIRS` setting in `C:\ProgramData\SD\sd.conf`, separating several with a semicolon. Nothing else needs changing. `config('NETDIRS')` prints what is in force.

THE PASSWORD FILE, THE PROGRAM CATALOGUE AND THE ACCOUNT REGISTER ARE NEVER REACHABLE from an API session, and cannot be added to `NETDIRS`.

An API session runs as you

Records an API session creates are owned by the account that logged in, and the session reaches files with your access rather than the service's. If your account may not read something, the API session may not read it either.

This was not true before 24 Aug 2026, and the way it failed is worth knowing because it was invisible: taking on your Windows identity applied only to the one thread that did it, and starting a short-lived helper process during the switch quietly put the session back to the service's own identity — **with no error and nothing in the log** — before it had opened a single file. So the login half worked and the part you would notice did not: records came out owned by the system account.

If your identity cannot be taken on, the login is now refused rather than continued with the service's identity. A session that believes it is you while holding the service's rights is worse than one that never started.

There is an alarm for the condition returning. If a session ever believes it is you while Windows says it is not, the error log gets:

```
API IDENTITY LOST at record write - session believes it is ...
```

Nothing is written when the two agree, so **a healthy session is silent** and you should never see this. It refuses and slows nothing; it exists so the condition cannot go unnoticed a second time.

`sh` and `OS.EXECUTE` are refused over the API

Before this port they were not refused, and on Windows that turned out to be worse than it sounds: **an API session ran as the LocalSystem account**, so a remote client could run any command on the machine with full privilege — more than the administrator sitting at the keyboard gets.

An API session is no longer treated as an administrator for any purpose, which is what SD's own code already assumed and did not enforce.

Client libraries

64-bit

`sdc1lib.dll`

32-bit

built separately; 32-bit remains a supported target, not a test convenience

Both must come from this release or later. A client library that predates SCRAM is refused by the server, and the 32-bit client in particular once shipped sending passwords in clear — check which one your application is actually loading.

Client distribution

Which library an application needs, why there are four names, and the one file no installer can update.

An application reaches SD Core through a client DLL. **You do not have to find it inside an installed SD system** — the clients are packaged separately, with their own installers, because the person writing an application and the person running the server are usually not the same person.

The four names, and why

Two builds, each producing its DLL under **two names from one source in one build**:

BUILD	NAMES	FOR
64-bit	<code>sdclilib.dll</code> · <code>sdclient.dll</code>	new work, and existing SD applications
32-bit	<code>qmclilib.dll</code> · <code>qmclient.dll</code>	QM applications, and mvDeveloper

The `*clilib` names are what existing applications ask for and they never move. `qmclilib.dll` in particular is the original QMClient library name — it is **how an unmodified QM application finds its client at all**, so it is a name rather than a label and it is not going to be renamed.

The `*client` names are for new work and are what the installers put on `PATH`.

They are second **links**, not file copies. An import library records the DLL name its symbols come from, so a renamed copy would send the application back to the original.

Both builds come from one source

`sd4windows\sdb_ai\sd64\gplsrc\sdclilib` is the single source of truth. The 32-bit build points straight at it.

There is deliberately no second copy. An earlier build of this port had a middle hop, and it let the 32-bit DLL go stale and **ship without SCRAM** — sending passwords in clear against a server that no longer accepted them. The hop was removed.

If you are testing the 32-bit client, confirm which build you have. A client that predates SCRAM is refused outright by this server, and the refusal reads as a wrong password.

Which one does my application need?

A 64-bit application	<code>sdclient.dll</code> , or <code>sdclilib.dll</code> if it already asks for that name
----------------------	---

A 32-bit application	<code>qmclient.dll</code> , or <code>qmcclilib.dll</code> if it already asks for that name
----------------------	--

mvDeveloper	<code>qmcclilib.dll</code> , 32-bit — and see the warning below
--------------------	---

The architecture must match the application, not the machine. A 32-bit application on 64-bit Windows needs the 32-bit DLL. **The 32-bit build is a shipping deliverable, not a testing convenience.**

The one file no installer can update

mvDeveloper is free, and is a 32-bit application — <https://www.brianleach.co.uk/mvDeveloper>.

It loads its own copy of the client, from:

```
C:\Program Files (x86)\BLC\mvDeveloper\qmcclilib.dll
```

That is **beside the executable, and Windows searches an executable's own directory before** `PATH` — so no installer entry and no `PATH` change will ever update it. It has to be replaced by hand.

This is the most likely way to test an old client without realising it. If mvDeveloper cannot log in after an upgrade, check that file's date before anything else.

The library must match the release

The cleartext API login is gone. A client that still sends a password in clear is refused with *"Cleartext login is no longer supported; this server requires SCRAM authentication"*.

Two things to do, and the second is the one people miss:

1. Use a client library from this release or later.
2. **Run** `modify.password` **again for every account that uses the API.** The stored credentials changed shape and the old ones cannot be converted — the password was never kept anywhere, by design, so there is nothing to convert them from.

An account whose password has not been re-set is refused, **and the refusal reads as a wrong password**, because from the server's point of view there is no credential to check.

Connecting

Your application code does not change. `SDConnect()` and `SDConnectLocal()` take the same arguments and return the same things they always did.

<code>SDConnect()</code>	over the network, to port 4243 — not an ssh tunnel any more
--------------------------	--

<code>SDConnectLocal()</code>	on the same machine. Sends no password and never did , so SCRAM does not apply
-------------------------------	---

BASIC programs reaching another SD server use the `!sdclient` class, which speaks the new login too — `connect()` takes the same arguments, but **the program has to be running under this release at both ends**.

Everything about what a connected session may open, the three gates it must clear, and the identity it runs as is on API access.

Building from source

There is one repository and it builds all four DLLs: <https://github.com/dmontaine/sd4windows>. The client source is in it, at `sdb_ai/sd64/gplsrc/sdclilib`, and `make sd` produces the 64-bit and 32-bit pairs together with the server. **No built DLL is committed**, so a clone builds — the same no-binaries rule the whole repository follows.

WHERE THE BUILT CLIENTS ARE AFTER AN INSTALL

The server installer places them under `C:\Program Files\SD`:

<code>usr\clients\client64\</code>	<code>sdclilib.dll</code> , <code>sdclient.dll</code> , and the import libraries to link against
------------------------------------	--

<code>usr\clients\client32\</code>	<code>qmclilib.dll</code> , <code>qmclient.dll</code> , and their import libraries
------------------------------------	--

<code>usr\bin\</code>	both DLL pairs again, beside <code>sd.exe</code> . That directory is on <code>PATH</code> , so it is where a 32-bit utility finds its client at run time
-----------------------	---

The C headers are not shipped yet. Take them from the repository.

A note on how the 32-bit client is built, because it constrains changes to it: the DLL must stay a **single self-contained file that can be copied next to an application** — hence static linking of the compiler runtime, and hence a preference for Windows' own `bcrypt.dll` and `crypt32.dll` over third-party crypto libraries. Any change to the client has to keep working in a 32-bit process.

How the clients will be published

THERE WILL BE A CLIENT INSTALLER OF ITS OWN, AND IT WILL CARRY MORE THAN THE DLLs.

In it	the client DLLs, this documentation, and the related utilities
Not in it	source code of any kind
Where the source is	GitHub only — https://github.com/dmontaine/sd4windows . The installer creates a <code>docs</code> subdirectory and names the repository there

That installer is not what W1.0-0 ships. It is a change to the installer and it has not been made yet. What exists today is what the section above describes: the server installer builds and places all four DLLs, and there is no client-only package.

So if you need a client and cannot find one, ask rather than building from source — the answer may be that a package is waiting.

Lower case

Commands, file names, record ids and account names are lower case now — and nothing you type has to change.

Everything that can be lower case is lower case. SD used to be inconsistent about it: BASIC source is free-form and usually written lower case, while file names, field names and account names were forced up.

What you type does not change. `LIST`, `list` and `List` all run the same verb, and so does every keyword — `with`, `by`, `no.page` and the rest.

The lookup rule

SD tries a name **as you typed it, then in lower case, then in upper case.**

```
as typed → lower → upper
```

A name that matches exactly still wins, so nothing that works today changes. A name that exists in **no** case is still reported as not found.

This order is used everywhere: the parser, the query processor, `RUN`, multifile resolution, the `LOGIN` paragraph lookup, and `SET.FILE`'s default `qfile` pointer.

The change was additive, not a flip. A `downcase` attempt was inserted into a chain that already tried as-typed and then upper. On a tree whose ids are all upper case the new attempt can never hit, so it changed no behaviour and could not break anything.

What is spelled in lower case now

Commands in the VOC

`list`, `count`, `select`, `create.file`, `setptr` ... and that is how they appear in `list voc`, `listv` and `ct voc`

System files on disk

`accounts`, `bp`, `bp.out`, `gpl.bp`, `messages`, `newvoc`, `pcode.out`, `syscom`, `voc`, `voc.dic`, `voc_template` and the rest

The `BP` and `GPL.BP` VOC entries

`bp` and `gpl.bp`

The hold file

`$hold`

The saved select list file

`$savedlists`

The command stack record

`$command.stack`

Account names on disk

`sdsys\accounts\don`, matching the account's own directory in
`user_accounts`

Files in a new account

created with lower-case names on disk

Renaming these is cosmetic for resolution — NTFS matches without being asked — but the stored path text is user-visible through `listf` and the current-directory reporting, which is the point.

Record ids in directory files are no longer case sensitive

Two changes that go together:

- **Record ids in directory files** are matched case insensitively.
- **Queries against a directory file** match ids the same way.

`create.file` also takes a `no.case` option, which creates a file whose record ids are treated as case insensitive: SD writes records preserving the casing given by whatever performs the write, and reads locate records regardless of casing.

One correction worth reading

`LIST` and `CT` used to disagree about the same name. `list voc $HOLD` answered *"'\$HOLD' not found"* on the very record `ct voc $HOLD` had just shown you. `LIST`, `SORT`, `SELECT` and the rest of the query language now use the same as-typed → lower → upper order as everything else.

Account names

Account names have never been case sensitive and still are not. `create.account`, `logto` and the rest accept whatever case you type.

What changed is only how the register file is named on disk: new accounts are recorded in lower case, so

`sdsys\accounts\don` matches the account's own directory. **Existing accounts keep the names they already have.**

Existing accounts are not touched

An account created before this keeps the upper-case names in its VOC and goes on working. there is nothing to migrate. An account you create now, or one you refresh with `update.accounts`, gets the new spelling.

Because SD only ever *adds* VOC records at an update, an old account will end up holding both spellings after `update.accounts`. That is harmless — they dispatch to the same programs.

The Turkish and Azeri fix

The installer creates an SD account for whoever authorises the install, and to do that it matches your Windows user name against SD's copy of it — which means changing both to the same case.

Windows and SD did not change case the same way everywhere. On a Turkish or Azeri system Windows turns `I` into a dotless `ı`, and SD does not. A user name containing that letter did not match itself, and the install finished **without giving you an SD account at all.**

Both sides now use the same rule, which does not vary by locale. Nothing changes on a system whose locale was never affected.

If you are testing on a Turkish or Azeri locale, this is worth exercising specifically — it is the kind of fault that only appears on the machine you do not have.

Two related refusals that no longer depend on case

`delete.file`'s refusal to delete `voc` and `$acc` no longer depends on the case you type. It could not be got round before, because those names were upper case — **it could have been, once they are not.**

Security

Who you are, what that gets you, and what actually protects the database.

The identity model in SD Core is not OpenQM's and not SD on Linux's. It is worth understanding before you test anything else, because several behaviours that look like bugs are consequences of it.

Signing in asks for no password

The operating system has already authenticated you. SD asks Windows who you are.

<code>sd</code> , no account named	you land in the SD account with your own name
no SD account of that name	refused — <i>Account %1 not in register</i> (5018)
not in <code>sdusers</code>	refused at the door — <i>not registered for SD use</i> (5009)
<code>sd -A<name></code>	refused unless <code><name></code> is your own account (10051)
an elevated session	your own account, like everybody else. <code>logto sdsys</code> afterwards
<code>logto sdsys</code>	asks for no password. The gate is elevation — a UAC consent prompt if the session is not already elevated, and 10002 with an audited <code>LOGTO REFUSED</code> if that fails

There is no SDSYS password. There is deliberately no second shared secret held by every administrator — that is the OpenQM weakness this exists to remove.

`sd <command>` **never asks you to set a password.** It runs and exits, so it is not a session anybody is being invited into. In earlier builds of this port it walked into a prompt it could never be given input for. See Running SD.

Being an administrator

If you can log in to Windows as an administrator, you are an administrator of sd. The person who installs SD is an SD administrator without any further step.

Two different questions are asked in two different places, and both are wanted:

QUESTION	HOW IT IS ANSWERED	GATES
<i>Are you an administrator?</i>	the account's groups in the SAM	<code>sd -start</code> — starting the server should not demand elevation of somebody already an administrator
<i>Are you elevated?</i>	the process token	reaching SDSYS , and every privileged action

A UAC-filtered token carries `Administrators` as *deny only*, so these give different answers for the same person, and conflating them is the easy mistake.

A third question is asked as well: *where did this session come from?* Being an administrator and being elevated are no longer enough on their own — the session also has to have started on this computer. An administrator signing in from another machine is refused, over ssh and over the API alike, whichever way the first two questions are answered.

One property to accept consciously. `Administrators` is machine-wide, so anyone in it for an unrelated reason — the machine's own administrator, a domain admin, an IT tool's service account — gets SDSYS. Linux sudoers is machine-wide too, so this is parity rather than a Windows weakness, but it should be a decision rather than a discovery.

Taking an account out of use without deleting it

`modify.account fred suspended` denies entry at all three ways in — ssh and the console, `logto`, and the API. It is reversible with nothing to remember: the VOC and every Windows group membership are left exactly as they are, and the tier it displaced is recorded, so naming a tier brings the account back where it was.

Understand what it is and is not, because the name oversells it.

It is	an SD control. The three doors SD owns are shut
It is not	a Windows control. Nothing is withdrawn there

So a suspended user's ssh connection is still accepted and SD still starts before refusing them, and **a suspended administrator keeps `Administrators`, keeps their `os.users` record, and can still elevate on this machine.** An elevated session can also still `logto` into a suspended account, which is deliberate — that is how you look at one.

If you are Suspending an account to contain somebody rather than to park it, disable the Windows account too. Everything on this page rests on Windows identity; a control that does not touch Windows cannot be the whole answer.

Understand what the security position rests on

NOTHING IN SD CHECKS A SECRET AT CONSOLE OR ssh LOGIN. ACCESS IS ENTIRELY OPERATING-SYSTEM GROUP MEMBERSHIP.

That is not a weakening. Every SD process opens the database directly, in your own process, under your own token. There is no data server standing between you and the files. So:

While SD runs as the invoking user, account passwords organise access; they do not secure it.

A password gate inside SD is not a file security boundary. The old password model implied one the filesystem never enforced. This states the real position instead of dressing it up.

Passwords still matter for the API, which is a separate door and does require one — see API access.

What actually protects the database

THE TREE IS PRIVATE FROM THE REST OF THE MACHINE

`C:\ProgramData` grants `BUILTIN\Users` read and execute by inheritance, so the Windows default is world-readable and snooping needs no privilege at all. The installer **breaks inheritance first** and grants narrowly: `SYSTEM`, `Administrators` and `sdusers`.

This is the step that makes the data private, and nothing at run time Substitutes for it.

YOUR ACCOUNT DIRECTORY IS LOCKED TO YOU

`C:\ProgramData\SD\user_accounts\<name>` is granted to that account's own group, to administrators and to the system, **and to nobody else**.

Before this, every SD user could read and rewrite every other user's account files from Explorer or a command prompt — outside SD, so none of SD's own permission checks applied. SD has always refused to let you `logto` an account you are not a member of; this makes the files agree with that.

Nothing you type changes, and your own account is unaffected — you are a member of its group.

A directory whose Windows group has been removed is **skipped deliberately** and named in the report. Locking it to a group that no longer exists would take the account away from the person who owns it.

THE SYSTEM DIRECTORIES ARE READ-ONLY TO ORDINARY USERS

Seven parts of the SD data area could be changed by anyone in `sdusers`. They are now readable by everyone who needs them and writable only by an administrator:

<code>sdsys\accounts</code>	the register of accounts	
<code>sdsys\map</code>	<code>sdsys\messages</code>	<code>sdsys\newvoc</code>
<code>sdsys\bp</code>	<code>sdsys\cat</code>	SD's own programs
<code>sd.conf</code>	the configuration read at start-up	

Nothing an ordinary user does needs to write them — not the spooler, not saved lists, not a phantom. The commands that *do* write them are already administrator commands and are unaffected.

`sdsys\ipc` is **deliberately unchanged**. Every session writes to it, and it is how a `phantom` is given its command.

The global catalogue and the pcode library are locked the same way.

An existing installation is not changed by an upgrade, because the installer never overwrites a data area that is already there. To apply it to one you already have, run this once per path from an elevated prompt:

```
powershell -File "C:\Program Files\SD\secure-sysdirs.ps1" -Path "C:\ProgramData\SD\sdsys\accounts"
```

THE CREDENTIAL FILE

ON INSTALLS MADE BEFORE 17 Aug 2026, EVERY SD USER COULD WRITE THE FILE SD KEEPS ACCOUNT PASSWORDS IN. No password is stored there — SD keeps a scrambled verifier that cannot be turned back into a password — but **being able to replace one was enough**: a user could put in a verifier for a password of their own choosing and sign in as somebody else, including through the API.

The installer was always meant to lock that file down and never did. The step ran, reported nothing, and had no effect, because of a quoting fault in how it was called. **Nothing in the install looked wrong.**

Reinstalling is what fixes it. To check without reinstalling, from an **ordinary** (not administrator) prompt:

```
icaccls "C:\ProgramData\SD\sdsys\${CRED}"
```

If that prints permissions, the file is still open. **If it says "Access is denied", it is protected — which is what you want.**

Continued in

Security and the operating system — reaching the operating system from inside SD, privileged work, and the audit trail.

Security and the operating system

Reaching the machine from inside SD, how privileged work is done, and the audit trail.

This page continues Security.

Reaching the operating system from inside SD

There are three ways out of SD onto the machine, and all three are on one list.

	WHAT IT IS	GOVERNED BY
<code>sh</code> and <code>!</code>	a shell at the <code>:</code> prompt	<code>os.users</code> field 1
<code>OS.EXECUTE</code>	the operating system from inside a BASIC program	<code>os.users</code> field 2
<code>edit</code> and <code>micro</code>	a text editor, running outside SD	<code>os.users</code> field 2

`os.users` is in SDSYS and is read-only to everybody else on disk. That ACL is the whole of the protection: without it a user grants themselves any of the three in one line, and every check above it is decoration.

Four rules hold for all three:

1. **A missing record, or a missing file, means no.** An installation that has never set `os.users` up denies all three to ordinary accounts. That is the opposite of the VOC tier lists, where a missing record means the *full* VOC — do not carry one convention across to the other.
2. **An elevated session passes on its own**, whatever the list says, so an empty list cannot lock the machine's own administrator out.

An ssh session *can* be elevated. Windows' OpenSSH runs as a system service and builds the logon token itself, so a member of `Administrators` arriving over ssh is handed a **full** token — not the filtered one a local sign-in would produce, and with nobody asked to consent.

It is why SD now refuses an administrator any session that did not come from this computer. Over ssh **on this machine** an administrator is still elevated, and still passes this rule without needing a record.

3. **An API session is never treated as an administrator — which is not the same as having no way out to the machine.** An API session never holds administrator rights and cannot reach SDSYS. But this list is consulted **by user name**, and it is consulted whether or not a session is elevated: an account whose `os.users` record says `yes` gets `sh` and `OS.EXECUTE` over the API exactly as it would anywhere else.

Every administrator has such a record, written when the account is created and refused to `modify.account os-off`. So an administrator's API session can genuinely run operating-system commands on the server — which is why an administrator is refused an API connection from any other computer. From this machine it works, and that is deliberate.

4. Nothing in SD limits what you then do. Once field 1 or field 2 says `yes`, SD is not standing between that person and the machine — **the boundary from that point on is their own Windows account's permissions**, and nothing else. A listed person gets a real shell, with pipes and redirection; an editor can open any file they are allowed to open, in the data tree or outside it.

So field 1 and field 2 are statements of trust in a person. They are not a convenience to be handed out because somebody asked for a full-screen editor. Grant them on the same basis you would grant a shell account on the machine, because that is close to what you are granting.

The record format, and how to grant and remove it, are on Administrator commands.

Privileged work is done through a script, not a command line

When SD creates an account, sets a password or edits a group, it writes a short script to a file and runs it, rather than putting a password on a command line where any local user could read it through Task Manager or WMI.

Those scripts go in `PSTMP`, a directory of its own inside the database directory, where each file belongs to the session that wrote it. Other SD users cannot read or change them.

Before that they went where every SD user could write, and the second consequence was the serious one: **another SD user could replace the script between SD writing it and SD running it, and their version would run with full administrator privilege.**

If `PSTMP` is missing, SD refuses the privileged work rather than falling back to the old location. You see the command fail rather than quietly running unprotected.

The audit trail

`C:\ProgramData\SD\sdsys\audit` records **every login, every refused login, every `logto`, every refused `logto`, and every `grant` and `revoke`**, with date, time and the Windows user it belonged to.

```
2026-08-16 11:42:07 user=don uid=1 pid=8624 LOGTO account=SDSYS
```

The refusals are the interesting half. An entry saying somebody who is not an administrator asked for SDSYS by name, or asked for an account they have not been granted, is the thing worth seeing. **Failed API logins are recorded too, with the reason.**

This is not the error log and does not behave like it. The error log throws away its oldest half when it fills; the audit file is **renamed with the date and time and a new one started, so nothing is ever discarded.** Removing the old ones is your decision — SD will not do it for you, and they will accumulate.

SD USERS CAN

SD USERS CANNOT

add to it

read it, change a record in it, empty it, rename or delete it

Windows itself refuses, not SD, so a user cannot quietly remove the line that records what they did. Administrators can read it and do anything else to it as well: this raises the floor against ordinary users rather than trying to constrain somebody who owns the machine.

Windows keeps its own separate record of account and group changes in the Security event log. **Read the two together.**

What is still not true

SD users are not isolated from each other's data at the file level, beyond the Per-account directory lock. Everyone who uses SD needs file access to the tree, because their own process does the I/O. Anyone deploying SD for ten people over ssh should be told that plainly.

SD has no file-level access control of its own on the console and ssh paths. The one place a path gate does exist is the API, where a session is confined to the account it stands in — see API access.

Other hardening

The catalogue and pcode locks, the logs, line endings, and the rest of the smaller changes.

Everything on this page is a change you may notice while testing, grouped by what it touches. The identity model and the file permissions are on Security; this is the remainder.

The global catalogue

Adding to or removing from the system-wide catalogue now requires administrator rights, whichever way you ask for it.

`catalog` already required them for the spelled-out form, `catalog bp myprog global`. **It did not require them for the form most people use** — putting a `*`, `!`, `_` or `$` in front of the name. `delete.catalog` required nothing at all, by either route.

This matters because the system-wide catalogue holds the programs SD runs for everybody, `$login` among them.

Replacing one ran your code in every session on the machine, administrators included; deleting one stopped everybody signing in.

Nothing changes for local and private cataloguing, which is what programmers use day to day:

```
catalog bp myprog      private catalogue, this account
catalog bp myprog local this account's VOC
```

Both still work, in any account you are allowed to `logto` into. The only thing an ordinary user can no longer do is catalogue a program whose name starts with `*`, `!`, `_` or `$` — those characters mean *system-wide*. Name it without one.

To catalogue system-wide you need an elevated session, not just an administrator account. Start SD from an elevated window and `logto sdsys`. Entering SDSYS from an ordinary window still gives you administrator rights *inside* SD, but **Windows itself now refuses the write** — so use the same elevated window you already need in order to recompile the system programs.

The pcode library

`<sysdir>\bin` holds the pcode library — the interpreter itself, which SD loads into shared memory at start-up and every session then runs.

UNTIL 23 Aug 2026 ANY MEMBER OF `sdusers` COULD WRITE TO IT, so one SD user could have replaced what everybody else's session executes, including an administrator's.

It is now readable by SD users and writable only by administrators. Nothing needs to write it after an install — only the process that starts SD reads it, and that is already elevated. **If you have anything that writes into `<sysdir>\bin`, it will now be refused and will need to run elevated.**

Scheduled jobs

A scheduled task can run an SD command without administrator rights, and only the commands an administrator has named for it. The permit list is the SDSYS file `batch.jobs`, locked read-only to SD users by the same control as the `os.users` list.

It has its own page: **Scheduled jobs**.

The logs

There are three, and they are not interchangeable.

FILE	WHERE	FOR
<code>audit</code>	<code>C:\ProgramData\SD\sdsys</code>	who did what — logins, refusals, <code>logto</code> , grants. See Security and the operating system
<code>errlog</code>	<code>C:\ProgramData\SD\sdsys</code>	diagnostics, and API connection records
<code>sd-elevate.log</code>	<code>C:\ProgramData\SD</code>	what the elevation helper actually did

`SD-ELEVATE.LOG`

It records when a helper started for a session, each script it was asked to run, the exit code that came back, and when it stopped.

Only administrators can read or write it. Ordinary SD users are not on its permissions at all. That is different from the audit trail, which SD users *can* add to because SD writes it as them; nothing unelevated ever writes this one.

It is a diagnostic, not the audit trail. For *who obtained privilege and when*, read `audit`. This file answers "*the account was not created — what actually happened*", which previously had no answer at all.

If the file is missing, nothing is logged and SD does not create one. That is deliberate: a log created on the fly would inherit permissions letting every SD user rewrite it, and **a record of privileged work that its own subjects can edit is worse than none**. A reinstall keeps whatever is already there.

THE ERROR LOG RECORDS WHO CONNECTS TO THE API PORT

Every accepted API connection adds a line naming the Windows process and account at the other end:

```
API connection from 127.0.0.1:59314 - pid 11448, GITORLI\don
```

Nothing is refused on the strength of it. This records who connected; it does not decide who may. The API's own checks are unchanged.

A connection forwarded over ssh shows `sshd`, not the person at the far end. The tunnel ends on this machine, so the process that connects genuinely is `sshd`. What the line distinguishes is a client running *on* this machine from one arriving through a tunnel; **it cannot name a remote person.**

This matters beyond the log. SD refuses an administrator an API session from another computer, and it decides that from the address the connection came from. A tunnelled connection arrives from `127.0.0.1` because it really does start here, so **it is admitted**. If that matters to you, turn port forwarding off in `sshd_config`; no check inside SD can see through a tunnel.

"peer process not identified" means the client had already gone by the time the connection was looked up. It is not an error and the connection proceeds normally.

TWO THINGS ABOUT ERROR-LOG TRIMMING

ERRLOG now applies to these lines too. The background daemon used to append without ever trimming — it only wrote at start-up and on failure, so it never grew. Now that it writes per connection, it discards the oldest part of the log on reaching the `ERRLOG` size in `sd.conf`. **If you have set `ERRLOG` unusually large, consider what an entry per connection adds to it.**

After the log is trimmed, its first line may have no timestamp. An entry is two lines — a timestamped header and the message indented below it — and trimming restarts the file at a **line**, not at an entry, so the first message can be left without its header. **This is not damage** and no entry after it is affected. SD has always trimmed this way; it is only visible now because the log turns over more often.

Line endings

Both halves are fixed, and they were fixed separately.

READING — FILES EDITED IN NOTEPAD OR SAVED FROM EXCEL

Directory files exist so you can edit their records with an ordinary Windows editor, and Windows editors end each line with CR+LF. **SD only ever looked for the LF, so it kept the CR — and put it on the end of the data.**

You would have seen it as an invisible extra character at the end of every line: comparisons failing for no visible reason, a name that would not match, a trailing space that was not a space.

Reading a CSV saved by Excel is the clearest case. The last column of every row picked up the stray character, because a comma ended the other columns and the line ending only ever touched the last one. `READCSV`, `READSEQ` and reading a directory file record are all corrected.

A CR on its own is still data and is left exactly as it is. Only the CR+LF pair that ends a line is treated as a line ending, so this cannot alter data that happens to contain a CR.

WRITING

Anything SD writes that an ordinary Windows program can open now ends its lines with CR+LF: records in a directory file, `WRITESEQ` and `WRITECSV` output, command output captured with `COMO`, printer output sent to a file, and the error log.

SD's CSV statements are documented as following RFC 4180, and **that standard asks for CR+LF**.

Dynamic files are unaffected — they are stored in SD's own format and are not readable by other programs.

Existing files are left alone, so a file can contain both endings. SD reads either, so this is untidy rather than a problem.

The terminal

The default terminal type is now `WINDOWS`. `TERM` on its own should say `Device : windows`.

THE ARROW KEYS DID NOTHING in cmd, PowerShell or Windows Terminal on earlier builds. A terminal has two spellings for an arrow key: in its ordinary state it sends `ESC [ D` for Left, and the other spelling `ESC O D` only after the application asks it to switch — **which SD never does**. The `vt100` definition SD was defaulting to lists only the second spelling, so SD was listening for a key no Windows console ever sends.

The shipped `WINDOWS` definition is an exact copy of `LINUX`, which had this right all along — its name describes an operating system, but what matters is the byte protocol.

Existing accounts keep their old setting until their VOC is updated. An upgrade now does that for every account, and `update.accounts` does it on demand. Until then, `term windows` sets it for the session. **63 definitions ship, compiling to 100 terminal names** — the extra names are variants such as `vt100-w` and `vt220-at` — so `term wyse60` still works.

A name that is not installed is refused and your current type is kept — "Unrecognised terminal name" — so a typo costs you nothing. `term` **with no argument reports the type actually in force**, which is how to check.

Watch for near-misses all the same. There is no plain `vt320` — the shipped name is `vt320-at`. `terminfo.src` ships with SD, so `sdtic` can add a definition that is not there.

Backspace works, at the prompt and when you are asked for a password.

THE PAGE IS 120 × 36, NOT 80 × 24

SD's default terminal size is 120 columns by 36 lines. It is not a cosmetic default: the shipped `@` dictionary records and the default `list` report layouts are formatted for 120 columns. **A console window narrower than that makes ordinary reports look wrapped or truncated**, which reads as a formatting bug and is not one.

`term` reports the size in force, above the `Device` line:

```
:term
Page width: 120
Page depth: 36
Device      : windows
```

The size is worked out at login, in this order: the `LINES` and `COLUMNS` environment variables if they are numeric, otherwise the terminfo entry's `lines` and `cols`, **otherwise 36 and 120** — then raised to a minimum of 10 × 20 if smaller. So a console or ssh session normally gets its real window size and 120 × 36 is the fallback when nothing answers, which is the case for a phantom or a piped script.

`term default` restores it, and it prints nothing when it does. It sets the same 120 × 36 the login path falls back to and returns silently, so run a bare `term` after it to see the result. `term 120,36` does the same by hand.

If you have notes from an earlier build, this is one of the things that changed: `term default` used to set 20 × 24 — the *minimum* width and a fixed depth rather than the defaults — so it made the display worse instead of putting it back.

Paths

A Windows PATH typed at the command prompt is no longer cut off at the first backslash. Typing `C:\Data\Sales` was read as just `C:`, with the rest treated as a second, separate thing. `create.account other`, which is given a folder to put the account in, was the command most likely to show it — the account went to the wrong place, or the command failed for a reason that made no sense from what you had typed.

Forward slashes always worked and still do. **Both are now read the same way.**

PowerShell execution policy — leave it alone

This is a hardening page, so it is worth saying plainly: **tightening your PowerShell execution policy does not break SD, and loosening it does not help SD.** Set it to whatever your own security policy wants.

SD does a lot of its Windows-side work — `logto sdsys`, `append.sd.path`, `remote.api`, `remote.ssh` and the editor verbs — by running a small PowerShell script that the installer put in `C:\Program Files\SD`. Each one is launched with `-ExecutionPolicy Bypass` on that single command line, which applies to that one process and that one script. It does not change the machine, and it does not affect any other script you or anyone else runs.

If you loosened the policy to get SD working, put it back. That was needed on builds before 5 September 2026, where those commands failed with *"running scripts is disabled on this system"*. It is fixed. The `sh` verb still gives you a PowerShell prompt under your machine's own policy — SD lifts the restriction only for the scripts it installed itself, never for a shell you type into.

The one case that does stop SD is Group Policy. A policy that sets the execution policy outranks anything a program can pass on a command line. Run `Get-ExecutionPolicy -List`: if `MachinePolicy` or `UserPolicy` reads `Restricted` or `AllSigned`, SD's administrative commands will fail and only your Windows administrator can change it. `Undefined`, `RemoteSigned`, `Unrestricted` or `Bypass` on those two rows are all fine. **The Installed Scripts page in the Administrator set has the detail.**

Running SD

The service	String Database (SD)
After an unclean shutdown	SD now starts anyway, rather than refusing because the last stop was abrupt
Nested sessions	SD will not start a second time inside itself
<code>sd <command></code>	needs an elevated session, or an entry in <code>batch.jobs</code> — see above

Setting no password

Earlier builds of this port had both the end of the installer and SD's own prompt say that without a password you could not use ssh or the API, and stop there. **That was true and easy to read as "some things will not work".**

It is stronger than that, and both now say so: with no password the account can be used **only at that computer** — at the keyboard, or through Remote Desktop or similar remote-control software — **and only from a session run as administrator.**

You can still choose it, and SD asks again the next time you open the account.

Not in SD Core

What has been removed, why, and what to use instead.

This page exists so you do not spend time hunting for something that is not there. **It names what is gone and what to use in its place; it does not document the removed features themselves.**

Everything here was in OpenQM, in ScarletDME, or in SD on Linux, and is not in SD Core for Windows.

If you had a use for any of these, say so. Several were removed on the reasoning that nothing needed them. That reasoning is worth testing against real use, and for some of them an administrator can put the verb back into an account's VOC — the programs behind a few of them are still installed.

Editors

GONE	USE INSTEAD
<code>sed</code> — the full-screen editor	<code>edit</code> , or <code>ed</code> for the line editor
<code>update.record</code> — the full-screen record editor	<code>edit</code> or <code>ed</code>
<code>modify</code> — the full-screen record editor from OpenQM	<code>edit</code> or <code>ed</code>

SD Core's own full-screen editing is `edit` and `micro`, which open the record in Microsoft Edit and in micro — see Programmer commands, which also says what they are good for and what they cannot do. The three above are gone as *programs*; the capability is not.

`modify` is received by **no account of any tier**.

`micro` **was on this page and has come off it — under its own name.** It was removed on 17 Aug 2026 because it launched an external editor, which is a way out of SD onto the machine underneath it. That was reversed on 26 Aug 2026, and there are now **two** full-screen editors, `edit` and `micro`: the same idea, done deliberately, limited to programmer and administrator accounts and refused over the API.

`modify.account` and `modify.password` **are not affected**. They are different verbs with different programs behind them, and both remain administrator commands.

`ed` was never affected by the keyboard faults that hit the full-screen editors — it reads whole lines and goes through the command-line editor. **If backspace is ever reported broken in `ed`, that is a new fault, not an old one returning.**

The PROC language

`PROC` is gone. **A VOC item of type `PQ` now reports that PROC is not supported instead of running.** Your `PQ` records are left alone — it is the interpreter that has gone, not the records.

Do not confuse PROC with the query processor. `LIST`, `COUNT`, `SELECT` and `SORT` are unaffected. They are a different thing despite the similar name.

SDNet — remote file access

SD could open a file held on another SD server by putting `server;file` in a VOC entry. **That is gone. A VOC entry containing a semicolon is now simply a file name that does not resolve.**

`SET.SERVER`, `DELETE.SERVER` and `LIST.SERVERS` have gone with it. **Two of those three had never worked in any case** — their VOC entries were malformed.

Why it went: each server's user name and password were kept in `sd.conf`, obscured with a simple letter substitution that is not encryption, and the session ran over port 4245. There was also no way to switch the feature off — the `NETFILES` setting was read at start-up and then never consulted.

The API is not affected. `SDClient` and the remote API are a separate mechanism and are unchanged. See API access.

`NETFILES` is still accepted in `sd.conf` and does nothing, so an existing configuration file will not stop SD starting.

Virtual file systems

SD has never been able to open a virtual file system. Nothing in the file-opening code ever recognised a `VFS:` pathname. What the language carried was the *outline* of one, and none of it could be reached: a VOC F-pointer written as `VFS:something` was reported as a virtual file system, passed the name resolver, and then failed to open with an unrelated error.

All of it has been removed, so the language no longer offers a feature it cannot perform.

These names are no longer defined, and a program mentioning one will no longer compile:

<code>FL\$TYPE.VFS</code>	<code>SYSCOM KEYS.H</code>
<code>ER\$VFS.NAME</code>	<code>SYSCOM ERR.H</code>
<code>ER\$VFS.CLASS</code>	<code>SYSCOM ERR.H</code>
<code>ER\$VFS.NGLBL</code>	<code>SYSCOM ERR.H</code>

`FTYPE` no longer returns `VFS` for a `VFS:` pathname. **Error numbers 3038, 3039 and 3040 are retired and will not be given a new meaning.**

IF ONE OF YOUR PROGRAMS REFERS TO ANY OF THESE, it was testing for a state SD could not reach, and the test can be deleted.

Two unreachable pieces went with it: `_EXTENDLIST`, installed into the SDSYS `GPL.BP` file and loaded at every start-up although nothing ever called it; and the debugger's `(Networked)` file type, which no file could report once SDNet was gone.

Language and locale

`NLS`, `SET.LANGUAGE` and `LOAD.LANGUAGE` are removed. **SD Core is English only**, and these were the only callers of the message-language machinery.

Embedded Python

Dropped, and it is a statement about what SD Core is for rather than a packaging choice: the intended use is as a back end data store reached through the API.

The C sources, the Makefile flags, 20 `GPL.BP/PY_*` programs, `SYSCOM/SDPYFUNC.H`, the `SD_Py*` error codes and the SDEXT keys have all gone.

Field-level encryption

`encrypt.field` is gone, and with it **field-level encryption from TCL**. The verb is in **no account's VOC at any tier** — it left `newvoc/TIER.ADD.ADMINISTRATOR` before W1.0-0. While it was still there it could not have worked: the `$CRYPTO` program behind it is not in the distribution, and every form of the verb failed at load, before it looked at what you typed.

Encryption in SD BASIC is unaffected and is the supported route. `sdencrypt()` and `sddecrypt()` ship — see *SD Basic - System and Environment* — and replaced the older `encrypt()` and `decrypt()` functions. What has gone is the TCL verb that encrypted a field in place, and **nothing replaces that**.

Account and configuration items

GONE	NOTES
<code>RDPACCOUNT</code> , <code>NO.RDPACCOUNT</code>	typing it now stops <code>create.account</code> with <i>Unexpected token (RDPACCOUNT)</i> and makes no account
<code>CREATUSR</code>	<code>config</code> no longer lists it; <code>config('CREATUSR')</code> returns nothing. A <code>CREATUSR</code> line in <code>sd.conf</code> is still accepted and ignored

`umask`

removed from every tier. It controls POSIX file-mode bits, which Windows does not use for security

Field 4 of an `ACCOUNTS` record the list of accounts allowed in. `list.grants` answers that question now

Accounts already created with `RDPACCOUNT` keep their Windows sign-in. Nothing goes round and takes it back, because SD did not record which accounts they were. If you have any, either delete and recreate them, or add them to the restricted group by hand:

```
net localgroup sdsshonly <name> /add
```

The five programs SD used to ship into the SDSYS BP file

`PCL` and `PCL.GRID` (printer control), `U0032` and `U50BB` (user exits) and `VFS.CLS` (a template class module) are no longer installed there.

PCL is unaffected as a printer feature — the `PCL` keyword and the catalogued `PCL` routine are both still there. What has gone is a second, older copy of the source sitting in `BP`.

SD now ships nothing into the SDSYS bp file. It is created empty and is yours — and because of that, **bp and its compiled objects are now preserved when you upgrade**, alongside your accounts and the rest of your own data.

Things that were never features, and are not coming

These are not removals. They are stated here because a reader coming from another MultiValue system will otherwise assume they exist.

Multi-user access over remote Desktop is not supported. It follows from the access model and is settled. One Windows setting covers Remote Desktop and the physical keyboard together, so allowing one allows the other. A verb that lifted the restriction was built and deleted the next day for exactly that reason. If you want it, you want Windows Server, RDP client access licences and probably a commercial product built for it. See ssh access.

SD cannot be installed silently. `/SILENT` and `/VERYSILENT` are refused, with a message saying why, and there is no switch to override it. Installing ends by asking for a password and a silent install has nobody to ask — it used to finish with **no password on any account** and say nothing about it. Unattended deployment is not supported.

scp AND sftp DO NOT WORK INBOUND ONCE SD HAS CONFIGURED THE ssh SERVER, for anybody, administrators included. This is the accepted cost of putting every ssh session straight into SD. **Pull files rather than pushing them** — see ssh access. A machine with no ssh server is unaffected, because SD has configured nothing.

The cleartext API login is gone, and a client that still sends a password in clear is refused outright. See API access.

Linux-only mechanisms

The Linux privilege model does not survive the move and has been replaced rather than emulated. Most of this is invisible unless you are reading source, but two consequences show:

- `chmod` **does nothing**. The MSYS2 mount is `noac1`, so file-mode bits are not a security control here. Windows ACLs are, and SD sets them.
- **There is no `sdsys` uid to drop to**. Privilege is elevation, and the operating system's groups are the whole of the authorisation model. See Security.